notebook

December 17, 2023

1 Log

1.1 Thomas

Date	Duration [h]	Task
18/11/2023	2	EDA: frame count and video count outlier detection
21/11/2023	2	Research motion extraction techniques as part of preprocessing pipeline
21/11/2023	2	Optical FLow: Farneback technique
27/11/2023	2	try original hand masking technique
27/11/2023	3	Optical Flow: Lucas Kanade technique
04/12/2023	4	Video length equalization technique
04/12/2023	2	Make methods to easily load and save (modified) datasets
06/12/2023	4	Refactor entire data laoding and preprocessing pipeline
07/12/2023	1	Completed pipeline to test kaggle data
08/12/2023	3	Compare different motion extraction techniques
09/12/2023	1	Provided documentation for entire EDA section
13/12/2023	2	Improved documentation for EDA and preprocessing
13/12/2023	1	Improved documentation
17/12/2023	1	Confusion matrix for CNN
18/12/2023	2	Finishing touches to documentation

1.2 Felix

Date	Duration [h]	Task
13/11/2023	3	implemented some preprocessing code into notebook
15/11/2023	5	Made the script that restructures the given dataset
16/11/2023	6	Made the script that calculates masks and saves them appropriately
18/11/2023	3	Tried implementing opticalflow
21/11/2023	3,5	reunion with Thomas about opticalflow and solutions
21/11/2023	3	made hand detection dataset based on hand segmentation
27/11/2023	3,5	reunion with Thomas and Cobe in lab class for general questions
27/11/2023	3	Did my best to implement a cnn network dint quite get the results
28/11/2023	7	implemented successful cnn network/trained it and wrote the code to test the kaggle competition set

Date	Duration [h]	Task
29/11/2023	5	wrote pipeline for testing kaggle cometition sets. something is still wrong.
6/12/2023	4	fixed an anoying bug in within the kaggle dataset
7/12/2023	3	tested ai parameters
9/12/2023	3	tested ai parameters
10/12/2023	6	documentation and testing ai parameters
13/12/2023	3	Tensorboard + documentation

1.3 Cobe

Date	Duration [h]	Task
04/12/2023	6	Added layers to rnn model & processed input data, started working on rnn prototype
05/12/2023	6	Tried to implement the correct data input for the RNN model + calls with Thomas & Felix
08/12/2023	5	Reimplemented data importation and feeding to rnn model using new functionaly, researched rnn layers
09/12/2023	8	Realised we weren't actually creating a rnn model, researched rnn models & keras/tf documentation, reimplemented rnn prototype & started working on rnn model V1
10/12/2023	4	Research (e.g. different optimizers) + Improved accuracy rnn model
11/12/2023	4	Made different RNN models, each with new features to improve the previous model + call with Felix to submit first rnn model to kaggle
13/12/2023	5.25	Only now realised how much the rnn models were actually overfitting, tried to fix it, failed
14/12/2023	4	Wrote documentation, inspected result outputs to spot where things go south, researched alternative rnn approaches
15/12/2023	5	Implemented CNN-RNN hybrid
15/12/2023	1.25	General lay-out & wrote documentation for RNN model
16/12/2023	4.25	Completed documentation of RNN models up to (but except the conclusion of) rnn_V3
17/12/2023	4	Completed documentation of all RNN models

2 Group 6 sprint-2

2.1 Dataset Preparation

2.1.1 Folder structure

Before analyzing the data, we need to restructure the dataset for simplicity and consistency.

Initially, the dataset had the following structure:

```
- root
    - poses (4 in total)
        -group name
            -video
                - depth
                - confidence
```

However, we aim for a more streamlined structure like this:

```
- root
    - poses (4 in total)
        -video
            - depth
            - confidence
```

The original folder layout was inconsistent, with some groups having excess folders like `\_\_MACOSX`
To transform the dataset to its desired structure, we used the following code:

```
def copy(path,paths:[]):
    print(f"paths = {paths}")
    print(f"paths0 = {paths[0]}")
    begin_counter = len(os.listdir(paths[0])) - 19
    for index,item in enumerate(os.listdir(path)):
        print(begin_counter)
        print(f"copying folder: {path}/{item} to {paths[0]}/video_{begin_counter+index} has nu
        shutil.copytree(f"{path}/{item}",f"{paths[0]}/video_{begin_counter+index}")


folders = os.listdir()
for folder in folders:
    if os.path.isdir(folder):
        for nextfolder in os.listdir(f"./{folder}"):
            if "video_1" in os.listdir(f"./{folder}/{nextfolder}"):
                copy(f"./{folder}/{nextfolder}",[folder,nextfolder])
            else:
                for lastfolder in os.listdir(f"./{folder}/{nextfolder}"):
                    print(f"lastfolder = {lastfolder}")
                    if lastfolder == "__MACOSX":
                        continue
                    copy(f"./{folder}/{nextfolder}/{lastfolder}",[folder,nextfolder,lastfolder
```

2.2 Importing python libraries

[1]:
```
# Python standard libraries
from dataclasses import dataclass
from datetime import datetime
import glob
import operator
```

```
import os
import re
from typing import Tuple
```

[2]:
```
# For data processing
import cv2
from IPython.display import HTML
import matplotlib
import matplotlib.animation as animation
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
```

[3]:
```
# For machine learning
import einops
import keras
from keras import metrics
from keras import layers
from keras.layers import Conv2D, Dense, Flatten, LSTM, TimeDistributed
from keras.models import Sequential
from keras.optimizers import Adam
from keras.regularizers import l2
from keras.utils import to_categorical
from sklearn.metrics import multilabel_confusion_matrix, confusion_matrix
import pandas as pd
import seaborn as sns
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
from keras import optimizers
```

2.3 Exploratory Data Analysis

2.3.1 Utility function to load and display dataset

[4]:
```
gesture_labels = {"Next": 0, "Rotateright": 1, "Scrolldown": 2, "Zoomin": 3}

# Increase max video size allowed in notebook
matplotlib.rcParams["animation.embed_limit"] = 2**30


def load_dataset(path: str, limit: int = None) -> list:
    """
    Loads the dataset from the given path and returns it as a tuple of lists.
    """
```

```
    X = []
    videos_path = sorted(os.listdir(path), key=numericalSort)
    for video_path in videos_path:
        video = load_confidence_video(os.path.join(path, video_path))
        X.append(video)
        if limit is not None and len(X) >= limit:
            break
    return X


def load_confidence_video(video_path: str) -> list:
    """
    Loads a confidence video from the given path and returns it as a list.
    """
    frames_path = os.path.join(video_path, "confidence")
    frames = []
    for frame_path in sorted(glob.glob(frames_path + "/confidence_*.npy")):
        frame = np.load(frame_path)
        frames.append(frame)
    return frames


def play_video(video_frames: list[np.ndarray]):
    """
    Takes a list of frames and plays it as a video using matplotlib animation.
    """
    frames_n = len(video_frames)
    fig, ax = plt.subplots()
    image_axes = ax.imshow(video_frames[0], cmap="gray")

    def update_mask(j):
        image_axes.set_array(video_frames[j])
        ax.set_title(f"Frame {j+1}/{frames_n}")
        return [image_axes]

    animated_hands = animation.FuncAnimation(
        fig, update_mask, frames=range(len(video_frames)), blit=True
    )
    plt.close()  # Avoid last frame to be plotted separately
    return HTML(animated_hands.to_jshtml())


numbers = re.compile(r"(\d+)")
def numericalSort(value):
    """
    Sorts the numerical part of a directory name numerically instead of␣
 ↪alphabetically.
```

```
    """
    parts = numbers.split(value)
    parts[1::2] = map(int, parts[1::2])
    return parts
```

The `numericalSort` function makes sure that the videos are read in the correct order. By default directories are read in the following way: - video_0 - video_1 - video_10

But we need it in the following sequence: - video_0 - video_1 - video_2

2.3.2 Distribution of videos per gesture and frames per video

We aim to identify anomalies such as unusually short or long videos by analyzing the number of frames in each video.

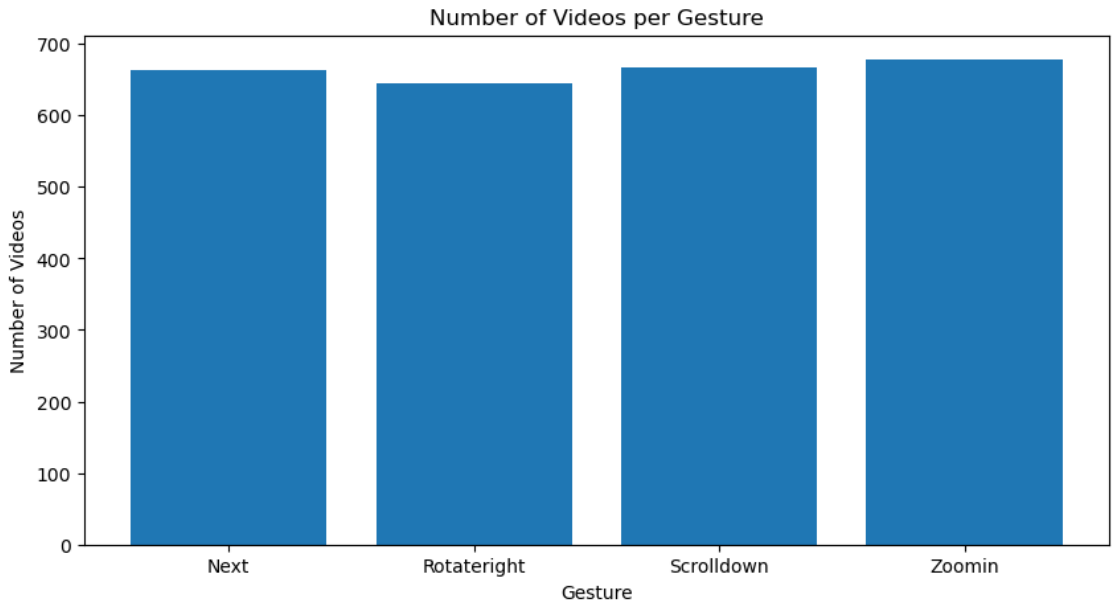
Additionally, to ensure balanced representation among the classification groups, we examine the distribution of videos across the different gesture classes.

[5]:
```
dataset_path = "../data/"

# Gather info about videos per gesture and frames per video
num_videos = {}
num_frames = {}

for folder in os.listdir(dataset_path):
    videos = os.listdir(dataset_path + folder)
    num_videos[folder] = len(videos)
    num_frames[folder] = {}
    for video in videos:
        frames = os.listdir(dataset_path + folder + "/" + video + "/confidence/
 ↪")
        num_frames[folder][video] = len(frames)
```

Plot of the distribution of videos per gesture

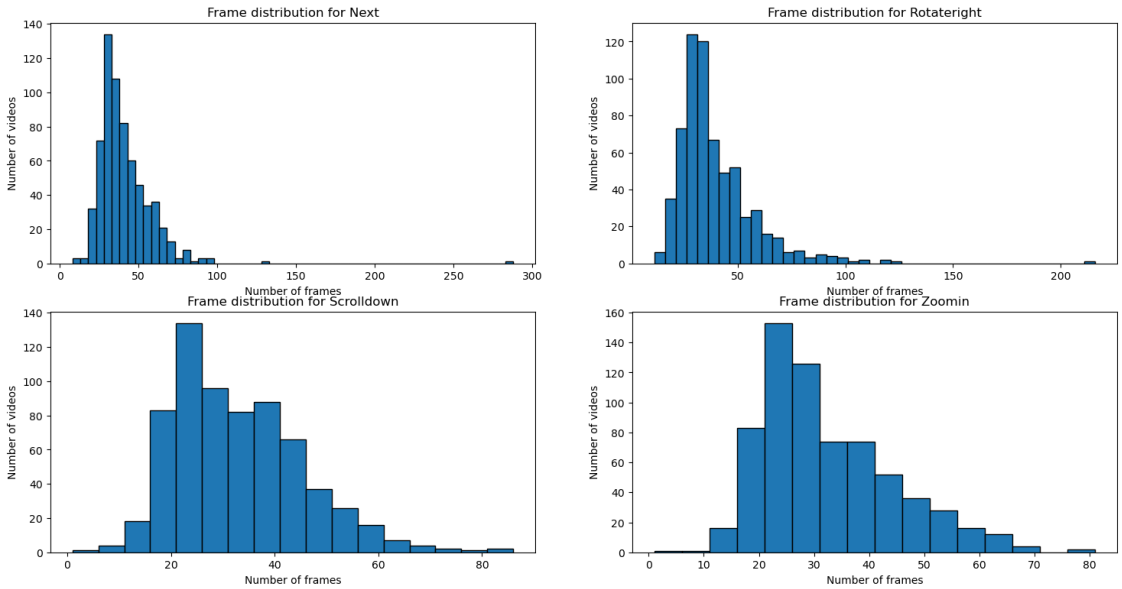
[6]:
```
print(num_videos)
plt.figure(figsize=(10, 5))
plt.bar(num_videos.keys(), num_videos.values())
plt.xlabel("Gesture")
plt.ylabel("Number of Videos")
plt.title("Number of Videos per Gesture")
plt.show()
```

```
{'Next': 664, 'Rotateright': 645, 'Scrolldown': 667, 'Zoomin': 678}
```



Plot of the number of frames per video

[7]:
```
binwidth = 5
plt.figure(figsize=(18, 9))
for i, gesture in enumerate(num_frames.keys()):
    plt.subplot(2, 2, i + 1)
    frame_counts = [int(count) for count in num_frames[gesture].values()]
    bins = np.arange(min(frame_counts), max(frame_counts) + binwidth, binwidth)
    plt.hist(frame_counts, bins=bins, edgecolor="black")
    plt.xlabel("Number of frames")
    plt.ylabel("Number of videos")
    plt.title(f"Frame distribution for {gesture}")
plt.show()
```



Inspect outliers To determine the validity of unusually small or large videos, we will conduct a visual inspection.

Example most frames in Next gesture In this particular video, the gesture itself only takes up around 30 frames. So this video on itself should count as bad data. We could either ignore this video, or trim it to the part where there is movement

[8]:
```
video_name_max_frames = max(num_frames["Next"].items(), key=operator.
↪itemgetter(1))[0]
video = load_confidence_video(dataset_path + "Next/" + video_name_max_frames +␣
↪"/")
play_video(video)
```

[8]: <IPython.core.display.HTML object>

Example fewest frames in Scrolldown gesture This specific video contains only a single frame, and as such, will be deemed invalid.

[9]:
```
video_name_min_frames = min(
    num_frames["Scrolldown"].items(), key=operator.itemgetter(1)
)[0]
video = load_confidence_video(
    dataset_path + "Scrolldown/" + video_name_min_frames + "/"
)
play_video(video)
```

[9]: <IPython.core.display.HTML object>

2.3.3 Testing different motion extraction techniques

In this sprint, we transition from static images to dynamic videos, where the exact hand shape is less critical for our model. Instead, we prioritize capturing the evolving movements within the video.

During this phase, we'll explore a range of motion extraction techniques, aiming to pinpoint the most effective one for integration into our preprocessing pipeline.

Optical flow: Farneback Optical flow is a method used in computer vision that tracks the movement of objects within a video by calculating the shift of pixels between consecutive frames.

Farneback is a specific type of optical flow. It visualizes the changes in pixel intensity between two successive frames for each pixel in the video, providing a detailed representation of motion.

[10]:
```
# Parameters for Farneback optical flow
farneback_params = dict(
    pyr_scale=0.5,  # To build the pyramids for each image; 0.5 is a typical␣
 ↪value.
    levels=10,  # Number of pyramid layers; 1 means that no extra layers are␣
 ↪created and only the original images are used.
    winsize=15,  # Averaging window size; larger values increase the algorithm␣
 ↪robustness to image noise and give more chances for fast motion detection,␣
 ↪but yield more blurred motion field.
    iterations=3,  # Number of iterations at each pyramid level.
    poly_n=5,  # Size of the pixel neighborhood used to find polynomial␣
 ↪expansion in each pixel; larger values mean that the image will be␣
 ↪approximated with smoother surfaces, yielding more robust algorithm and more␣
 ↪blurred motion field.
    poly_sigma=1.2,  # Standard deviation of the Gaussian that is used to␣
 ↪smooth derivatives used as a basis for the polynomial expansion; for␣
 ↪poly_n=5, you can set poly_sigma=1.1, for poly_n=7, a good value would be␣
 ↪poly_sigma=1.5.
    flags=cv2.OPTFLOW_FARNEBACK_GAUSSIAN  # Use the Gaussian filter instead of␣
 ↪a box filter of the same size for optical flow estimation; usually, this␣
 ↪option gives z more accurate flow than with a box filter, at the cost of␣
 ↪lower speed.
)


def get_farneback_frames(frames: list, background_treshold: float = 0.05):
    flow_frames = []
    prev_frame = frames[0]
    for frame in frames[1:]:
        flow = make_background_mask(cv2.calcOpticalFlowFarneback(prev_frame,␣
 ↪frame, None, **farneback_params), background_treshold)
        flow_frames.append(remove_background_noise(frame, flow))
        prev_frame = frame
```

```
    return flow_frames

def make_background_mask(frame: list, threshold: float = 0.05) -> list:
    """
    This generates a mask for the frame based of the magnitude of the optical␣
 ↪flow.
    """
    mag, _ = cv2.cartToPolar(frame[..., 0], frame[..., 1])
    mag = cv2.normalize(mag, None, 0, 1, cv2.NORM_MINMAX)
    mask = (mag > threshold).astype(int)
    return mask.astype(np.uint8)  # Convert mask to 8-bit single-channel image

def remove_background_noise(frame: list, mask: list) -> list:
    """
    This function takes a frame and a binary mask as input. The mask should␣
 ↪have the same dimensions as the frame.
    The function applies the mask to the frame, effectively removing the␣
 ↪background noise. The masked frame, where
    the background noise has been removed, is returned.
    """
    return cv2.bitwise_and(frame, frame, mask=mask)
```

Optical flow: Lucas Kanade Unlike Farneback, the Lucas Kanade technique emphasizes the change in only a few points in the video. These points are chosen by a corner detection algorithm. Ideally, this technique follows certain points, such as fingertips, so that the movement of the hand can be easily deduced.

[11]:
```
feature_params = dict( maxCorners = 6,
                       qualityLevel = 0.3,
                       minDistance = 2,
                       blockSize = 7)

lk_params = dict( winSize  = (15,15),
                  maxLevel = 3,
                  criteria = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,␣
 ↪10, 0.03))


def normalize_frames(frames):
    normalized = []
    for frame in frames:
        frame = cv2.normalize(frame, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.
 ↪CV_8U)
        normalized.append(frame)
    return normalized
```

```
def get_lk_frames(frames: list, feature_params: dict, lk_params: dict) -> list:
    lk_flow_frames = []
    prev = frames[0]
    prev_corners = cv2.goodFeaturesToTrack(prev, mask=None, **feature_params)
    for frame in frames[1:]:
        new_corners, status, error = cv2.calcOpticalFlowPyrLK(prev, frame,␣
 ↪prev_corners, None, **lk_params)
        good_new = new_corners[status==1]
        good_old = prev_corners[status==1]
        mask = np.zeros_like(frame)
        for i in range(len(good_new)):
            a, b = good_new[i].ravel()
            c, d = good_old[i].ravel()
            mask = cv2.line(mask, (int(a),int(b)),(int(c),int(d)), [0,0,255], 2)
            frame = cv2.circle(frame,(int(a),int(b)), 3,[255,0,0],-1)
        img = cv2.add(frame,mask)
        lk_flow_frames.append(img)
        prev = frame.copy()
        prev_corners = good_new.reshape(-1,1,2)
    return lk_flow_frames
```

Median frame differencing The median frame differencing technique is employed to eliminate the background from a sequence of video frames. This method calculates the median difference between consecutive frames, which helps to isolate the moving objects in the video. The use of the median makes this technique more robust to noise, ensuring a more accurate background subtraction.

[12]:
```
def frame_difference(frames: list) -> list:
    medianFrame = np.median(frames, axis=0).astype(dtype=np.uint8)
    gray_medianFrame = cv2.normalize(medianFrame, None, 0, 255, cv2.
 ↪NORM_MINMAX, dtype=cv2.CV_8U)
    diffs = []
    for frame in frames:
        frame = cv2.normalize(frame, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.
 ↪CV_8U)
        dframe = cv2.absdiff(frame, gray_medianFrame)
        _, dframe = cv2.threshold(dframe, 70, 255, cv2.THRESH_BINARY)
        diffs.append(dframe)
    return diffs


def get_median_difference_frames(frames: list, group_size: int = 5) -> list:
    diffs = []
    for i in range(0, len(frames), group_size):
        foo = frame_difference(frames[i:i+group_size])
        diffs.extend(frame_difference(foo))
    return diffs
```

Frame differencing This technique takes the focus away from the background by subtracting two consecutive frames with each other. This way, only the pixels that have changed (indicating movement) will be visible, effectively minimizing the impact of the static background.

[13]:
```
def get_video_diff(video: list) -> list:
    """
    Returns a video consisting of the difference between consecutive frames.
    """
    depth_diff = []
    for i in range(len(video) - 1):
        depth_diff.append(video[i + 1] - video[i])
    return depth_diff
```

Showcasing preprocessing techniques All four movement extraction techniques appear to be effective when visually inspected by humans. However, both optical flow methods exhibit a degree of sensitivity depending on the specific video used for demonstration, making them less trustworthy.

In terms of frame differencing, the technique of simply subtracting consecutive frames seems to yield superior results for model training. The median frame differencing technique, while effective in isolating movement, may be overly aggressive in data reduction, potentially discarding valuable information. Therefore, using the straightforward frame subtraction method could be more beneficial for our purposes.

Because of this we will incorporate the regular frame differencing technique in our preprocessing pipeline. Another advantage of this technique is that it is the leat computationally expensive.

The idea to test optical flow, along with some code examples, came from the following article: https://nanonets.com/blog/optical-flow/

[14]:
```
video_to_display = load_confidence_video('../data/Rotateright/video_333/')

farneback_video = get_farneback_frames(normalize_frames(video_to_display),␣
 ↪background_treshold = 0.4)
lk_video = get_lk_frames(normalize_frames(video_to_display), feature_params,␣
 ↪lk_params)
med_diff_video = get_median_difference_frames(video_to_display)
diff_video = get_video_diff(video_to_display)

fig, ax = plt.subplots(2, 2, figsize=(9, 7))
ax[0][0].set_title("Farneback")
ax[0][1].set_title("Lucas Kanade")
ax[1][0].set_title("median frame differencing")
ax[1][1].set_title("frame differencing")

image_axes1 = ax[0][0].imshow(farneback_video[0], cmap="gray")
image_axes2 = ax[0][1].imshow(lk_video[0])
image_axes3 = ax[1][0].imshow(med_diff_video[0], cmap="gray")
image_axes4 = ax[1][1].imshow(diff_video[0], cmap="gray")
axes = [image_axes1, image_axes2, image_axes3, image_axes4]
```

```
def update_plot(j):
    axes[0].set_array(farneback_video[j])
    axes[1].set_array(lk_video[j])
    axes[2].set_array(med_diff_video[j])
    axes[3].set_array(diff_video[j])
    return axes

ani = animation.FuncAnimation(fig, update_plot, frames=range(len(diff_video)),␣
 ↪blit=True)
plt.close() # Avoid last frame to be plotted separately
HTML(ani.to_jshtml())
```

[14]: <IPython.core.display.HTML object>

2.4 Preprocessing

The preprocessing pipeline is structured as follows: - Each frame is resized to a resolution of 100x100, effectively reducing the overall image size. - The frame differencing method is employed as the movement extraction technique. - To ensure uniformity, all videos are adjusted to the same length through either downsampling or upsampling.

[15]:
```
@dataclass
class OutputShape:
    num_of_videos_per_gesture: int = None
    video_length: int = 35
    x_dim: int = 100
    y_dim: int = 100


def preprocess_video(video: list, output_shape: OutputShape) -> list:
    """
    Preprocesses a video and returns the preprocessed video.
    """
    # Reduce the resolution of each frame
    resized_video = []
    if output_shape.x_dim != None and output_shape.y_dim != None:
        for frame in video:
            resized_frame = cv2.resize(frame, (output_shape.x_dim, output_shape.
 ↪y_dim))
            resized_video.append(resized_frame)

    # make new video consisting of the difference between consecutive frames
    diff_video = get_video_diff(resized_video or video)

    if output_shape.video_length is None:
```

```python
        return diff_video

    # equalize length of each video
    new_vid = video_to_fixed_length(diff_video, output_shape.video_length)

    return new_vid


def video_to_fixed_length(video: list, desired_length=35):
    """
    Downsamples or upsamples a video to the desired length.
    """
    length = len(video)
    if length < desired_length:     # Upsample
        return upsample(video, length, desired_length)
    elif length > desired_length:   # Downsample
        return downsample(video, length, desired_length)
    else:   #Keep video as is
        return video


def upsample(video: list, current_length: int, desired_length: int) -> list:
    """
    Upsample video by repeating some frames until desired length is reached
    """
    if current_length >= desired_length:
        return video
    ratio = current_length / desired_length
    upsampled_video = []
    for i in range(desired_length):
        index = int(i * ratio)  # index from original video
        upsampled_video.append(video[min(index, current_length - 1)])
    return upsampled_video


def downsample(video: list, current_length: int, desired_length: int) -> list:
    """
    Downsample video by removing frames until desired length is reached
    """
    if current_length <= desired_length:
        return video
    ratio = current_length / desired_length
    downsampled_video = []
    for i in range(desired_length):
        index = int(i * ratio)  # index from original video
        downsampled_video.append(video[min(index, current_length - 1)])
    return downsampled_video
```

```
def save_confidence_video(video: list, directory: str, gesture: str, video_name:
 ↪ str):
    """
    Saves a confidence video to the given path.
    """
    frames_path = os.path.join(directory, gesture, video_name, "confidence")
    os.makedirs(frames_path, exist_ok=True)
    for i, frame in enumerate(video):
        frame_path = os.path.join(frames_path, f"confidence_{str(i).zfill(12)}.
 ↪npy")
        np.save(frame_path, frame)
```

2.4.1 Video after preprocessing

[16]:
```
original_video = load_confidence_video("../data/Zoomin/video_0/")
preprocessed_video = preprocess_video(original_video, OutputShape(None, 35,␣
 ↪100, 100))
play_video(preprocessed_video)
```

[16]:
```
<IPython.core.display.HTML object>
```

Save preprocessed data for later use It's imporant to note that during this phase, we also filter out bad data. Videos that are either less than 10 frames or exceed 100 frames in length are excluded from the saved dataset.

[17]:
```
def is_valid_length(length: int, min_length: int = 10, max_length: int = 100)␣
 ↪-> bool:
    return not (length <= min_length or length >= max_length)


def save_dataset(input_path: str, output_path: str, output_shape: OutputShape):
    for gesture in gesture_labels:
        print(f"Preprocessing {gesture}...")
        gesture_path = os.path.join(input_path, gesture)
        for video_path in sorted(glob.glob(gesture_path + "/*")):
            video = load_confidence_video(video_path)
            if not is_valid_length(
                len(video) + 1
            ):  # +1 because video will lose one frame during preprocessing
                print(f"video: {video_path} has invalid length: {len(video) +␣
 ↪1}")
                continue
            video = preprocess_video(video, output_shape)
            save_confidence_video(
                video, output_path, gesture, os.path.basename(video_path)
```

```
            )
```

[ ]:
```
RAW_DATASET = "./archive/"
NEW_DATASET_PATH = "./data_preprocessed_100x100_35/"
shape = OutputShape(None, 35, 100, 100)

save_dataset(RAW_DATASET, NEW_DATASET_PATH, shape)
print(f"---\nPreprocessing finished successfully")
```

2.5 Training data preparation

2.5.1 Load saved dataset

The training data is loaded into two lists. The first list contains all the preprocessed videos, while the corresponding labels for each video are stored in the second list.

In this step, we also ensure balanced representation of each class by limiting the number of videos loaded per gesture to match the gesture with the fewest videos.

[18]:
```
DATASET_PATH = "./data_preprocessed_100x100_35/"
# VIDEO_COUNT_PER_GESTURE = 16
VIDEO_COUNT_PER_GESTURE = min(  # Limit to the smallest number of videos per␣
 ↪gesture
    [len(os.listdir(os.path.join(DATASET_PATH, gesture))) for gesture in␣
 ↪gesture_labels]
)
videos = []
labels = []

for gesture in gesture_labels:
    print(f"Loading {gesture}...")
    X = load_dataset(os.path.join(DATASET_PATH, gesture),␣
 ↪limit=VIDEO_COUNT_PER_GESTURE)
    videos.extend(X)
    labels.extend([gesture_labels[gesture]] * len(X))

print("---\nLoading was successful")
```

```
Loading Next…
Loading Rotateright…
Loading Scrolldown…
Loading Zoomin…
---
Loading was successful
```

2.5.2 Split into train, test and validation dataset

In this example a ration of 60/20/20 is used for train, test and validation consecutively.

[19]:
```
X_train, X_test, y_train, y_test = train_test_split(
    videos, labels, test_size=0.2, random_state=42
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.25, random_state=42
)

X_train = np.array(X_train)
X_test = np.array(X_test)
X_val = np.array(X_val)

y_train = np.array(y_train)
y_test = np.array(y_test)
y_val = np.array(y_val)
```

[20]:
```
print("(video_count, frames_per_video, (frame_dimensions))")
print("---")

print(f"{X_train.shape=}")
print(f"{y_train.shape=}")

print()

print(f"{X_test.shape=}")
print(f"{y_test.shape=}")

print()

print(f"{X_val.shape=}")
print(f"{y_val.shape=}")
```

```
(video_count, frames_per_video, (frame_dimensions))
---
X_train.shape=(1530, 35, 100, 100)
y_train.shape=(1530,)

X_test.shape=(511, 35, 100, 100)
y_test.shape=(511,)

X_val.shape=(511, 35, 100, 100)
y_val.shape=(511,)
```

2.6 Model building

2.6.1 Tensorboard / callbacks

We add our callbacks for our model here to the callbacks array.

use: `tensorboard --logdir ./tensorboard_logs`

to view your tensorboard

[21]:
```
class AccuracyReachedValueCallback(tf.keras.callbacks.Callback):
    def __init__(self, threshold):
        super(AccuracyReachedValueCallback, self).__init__()
        self.threshold = threshold

    def on_epoch_end(self, epoch, logs=None):
        val_acc = logs.get('accuracy')
        if val_acc is not None and val_acc >= self.threshold:
            self.model.stop_training = True
```

[22]:
```
train_id = datetime.now().strftime("%Y%m%d-%H%M%S") + "_posetrain"
tensorboard_dir = 'tensorboard_logs\\' + train_id

early_stopping_cb = tf.keras.callbacks.EarlyStopping(monitor='val_loss',␣
 ↪min_delta=0, restore_best_weights=True, patience=7)
tensorboard_log_cb = tf.keras.callbacks.TensorBoard(log_dir=tensorboard_dir,␣
 ↪histogram_freq=1, write_graph=True)
accuracy_reached_cb = AccuracyReachedValueCallback(0.99)

callbacks = [early_stopping_cb,tensorboard_log_cb, accuracy_reached_cb]
```

2.6.2 Confusion matrix

With the help of SKlearn, we can conveniently visualize a confusion matrix directly within a Kaggle notebook. This approach is less cumbersome compared to using Tensorboard, which isn't directly compatible with Kaggle. The following functions provide a streamlined implementation of this metric:

[23]:
```
def get_pred(X_test, model):
    y_pred = model.predict(X_test)
    y_pred = np.argmax(y_pred, axis=1)
    print(y_pred.shape)
    return y_pred

def show_confusion_matrix(X_test, model, relative: bool = False):
    y_pred = get_pred(X_test, model)

    # Compute the confusion matrix
    mcm = multilabel_confusion_matrix(y_test, y_pred)

    # Create a 2x2 grid of subplots
    fig, axs = plt.subplots(2, 2, figsize=(10, 10))

    # Flatten the axes array so we can easily iterate over it
```

```
    axs = axs.flatten()

    # Display the confusion matrix for each class
    for i, (cm, label, ax) in enumerate(zip(mcm, gesture_labels.keys(), axs)):
        if(relative):
            cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
            cm = np.around(cm, decimals=2)
            sns.heatmap(cm, annot=True, fmt='.2%', cmap='Blues', ax=ax)
        else:
            sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax)

        ax.set_title(f'Confusion Matrix for class: {label}')
        ax.set_xlabel('Predicted')
        ax.set_ylabel('Truth')

    # Remove unused subplots
    if i+1 < len(axs):
        for ax in axs[i+1:]:
            fig.delaxes(ax)

    plt.tight_layout()
    plt.show()
```

2.7 CNN model

[24]:
```
class Conv2Plus1D(keras.layers.Layer):
    def __init__(self, filters, kernel_size, padding):
        """
        A sequence of convolutional layers that first apply the convolution␣
↪operation over the
        spatial dimensions, and then the temporal dimension.
        """
        super().__init__()
        self.seq = keras.Sequential(
            [
                # Spatial decomposition
                layers.Conv3D(
                    filters=filters,
                    kernel_size=(1, kernel_size[1], kernel_size[2]),
                    padding=padding,
                ),
                # Temporal decomposition
                layers.Conv3D(
                    filters=filters, kernel_size=(kernel_size[0], 1, 1),␣
↪padding=padding
```

```
                ),
            ]
        )

    def call(self, x):
        return self.seq(x)


class ResidualMain(keras.layers.Layer):
    """
    Residual block of the model with convolution, layer normalization, and the
    activation function, ReLU.
    """

    def __init__(self, filters, kernel_size):
        super().__init__()
        self.seq = keras.Sequential(
            [
                Conv2Plus1D(filters=filters, kernel_size=kernel_size,␣
 ↪padding="same"),
                layers.LayerNormalization(),
                layers.ReLU(),
                Conv2Plus1D(filters=filters, kernel_size=kernel_size,␣
 ↪padding="same"),
                layers.LayerNormalization(),
            ]
        )

    def call(self, x):
        return self.seq(x)


class Project(keras.layers.Layer):
    """
    Project certain dimensions of the tensor as the data is passed through␣
 ↪different
    sized filters and downsampled.
    """

    def __init__(self, units):
        super().__init__()
        self.seq = keras.Sequential([layers.Dense(units), layers.
 ↪LayerNormalization()])

    def call(self, x):
        return self.seq(x)
```

```python
def add_residual_block(input, filters, kernel_size):
    """
    Add residual blocks to the model. If the last dimensions of the input data
    and filter size does not match, project it such that last dimension matches.
    """
    out = ResidualMain(filters, kernel_size)(input)

    res = input
    # Using the Keras functional APIs, project the last dimension of the tensor␣
↪to
    # match the new filter size
    if out.shape[-1] != input.shape[-1]:
        res = Project(out.shape[-1])(res)

    return layers.add([res, out])


class ResizeVideo(keras.layers.Layer):
    def __init__(self, height, width):
        super().__init__()
        self.height = height
        self.width = width
        self.resizing_layer = layers.Resizing(self.height, self.width)

    def call(self, video):
        """
        Use the einops library to resize the tensor.

        Args:
          video: Tensor representation of the video, in the form of a set of␣
↪frames.

        Return:
          A downsampled size of the video according to the new height and width␣
↪it should be resized to.
        """
        # b stands for batch size, t stands for time, h stands for height,
        # w stands for width, and c stands for the number of channels.
        old_shape = einops.parse_shape(video, "b t h w c")
        images = einops.rearrange(video, "b t h w c -> (b t) h w c")
        images = self.resizing_layer(images)
        videos = einops.rearrange(images, "(b t) h w c -> b t h w c",␣
↪t=old_shape["t"])
        return videos
```

[25]:
```
HEIGHT = 100
WIDTH = 100
VIDEO_LENGTH = 35

input_shape = (None, VIDEO_LENGTH, HEIGHT, WIDTH, 1)
input = layers.Input(shape=(input_shape[1:]))
x = input

x = add_residual_block(x, 16, (3, 7, 7))
x = layers.MaxPooling3D(pool_size=(2, 2, 2))(x)
print(x.shape)

x = add_residual_block(x, 32, (3, 7, 7))
x = layers.MaxPooling3D(pool_size=(2, 2, 2))(x)
print(x.shape)

x = add_residual_block(x, 64, (3, 3, 3))
x = layers.MaxPooling3D(pool_size=(2, 2, 2))(x)
print(x.shape)

x = add_residual_block(x, 128, (3, 3, 3))
x = layers.MaxPooling3D(pool_size=(2, 2, 2))(x)
print(x.shape)

x = add_residual_block(x, 256, (3, 3, 3))
x = layers.MaxPooling3D(pool_size=(2, 2, 2))(x)
print(x.shape)

x = add_residual_block(x, 128, (3, 3, 3))
print(x.shape)

x = layers.GlobalAveragePooling3D()(x)
print(x.shape)

x = layers.Flatten()(x)
print(x.shape)

x = layers.Dense(4)(x)
print(x.shape)

model = keras.Model(input, x)
```

```
(None, 17, 50, 50, 16)
(None, 8, 25, 25, 32)
(None, 4, 12, 12, 64)
(None, 2, 6, 6, 128)
(None, 1, 3, 3, 256)
(None, 1, 3, 3, 128)
```

```
(None, 128)
(None, 128)
(None, 4)
```

[26]:
```python
frames = next(iter(X_train))
model.build(frames)
model.summary()
```

```
Model: "model"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 input_1 (InputLayer)           [(None, 35, 100, 10  0           []
                                0, 1)]

 project (Project)              (None, 35, 100, 100  64
['input_1[0][0]']
                                , 16)

 residual_main (ResidualMain)   (None, 35, 100, 100  14992
['input_1[0][0]']
                                , 16)

 add (Add)                      (None, 35, 100, 100  0
['project[0][0]',
                                , 16)
'residual_main[0][0]']

 max_pooling3d (MaxPooling3D)   (None, 17, 50, 50,   0           ['add[0][0]']
                                16)

 project_1 (Project)            (None, 17, 50, 50,   608
['max_pooling3d[0][0]']
                                32)

 residual_main_1 (ResidualMain)  (None, 17, 50, 50,   81664
['max_pooling3d[0][0]']
                                32)

 add_1 (Add)                    (None, 17, 50, 50,   0
['project_1[0][0]',
                                32)
'residual_main_1[0][0]']

 max_pooling3d_1 (MaxPooling3D)  (None, 8, 25, 25, 3  0          ['add_1[0][0]']
                                2)
```

```
 project_2 (Project)            (None, 8, 25, 25, 6  2240
['max_pooling3d_1[0][0]']
                                4)

 residual_main_2 (ResidualMain)  (None, 8, 25, 25, 6  80384
['max_pooling3d_1[0][0]']
                                4)

 add_2 (Add)                    (None, 8, 25, 25, 6  0
['project_2[0][0]',
                                4)
'residual_main_2[0][0]']

 max_pooling3d_2 (MaxPooling3D)  (None, 4, 12, 12, 6  0          ['add_2[0][0]']
                                4)

 project_3 (Project)            (None, 4, 12, 12, 1  8576
['max_pooling3d_2[0][0]']
                                28)

 residual_main_3 (ResidualMain)  (None, 4, 12, 12, 1  320512
['max_pooling3d_2[0][0]']
                                28)

 add_3 (Add)                    (None, 4, 12, 12, 1  0
['project_3[0][0]',
                                28)
'residual_main_3[0][0]']

 max_pooling3d_3 (MaxPooling3D)  (None, 2, 6, 6, 128  0          ['add_3[0][0]']
                                )

 project_4 (Project)            (None, 2, 6, 6, 256  33536
['max_pooling3d_3[0][0]']
                                )

 residual_main_4 (ResidualMain)  (None, 2, 6, 6, 256  1280000
['max_pooling3d_3[0][0]']
                                )

 add_4 (Add)                    (None, 2, 6, 6, 256  0
['project_4[0][0]',
                                )
'residual_main_4[0][0]']

 max_pooling3d_4 (MaxPooling3D)  (None, 1, 3, 3, 256  0          ['add_4[0][0]']
                                )
```

```
 project_5 (Project)            (None, 1, 3, 3, 128  33152
['max_pooling3d_4[0][0]']
                                )

 residual_main_5 (ResidualMain)  (None, 1, 3, 3, 128  541696
['max_pooling3d_4[0][0]']
                                )

 add_5 (Add)                    (None, 1, 3, 3, 128  0
['project_5[0][0]',
                                )
'residual_main_5[0][0]']

 global_average_pooling3d (Glob  (None, 128)         0           ['add_5[0][0]']
 alAveragePooling3D)

 flatten (Flatten)              (None, 128)          0
['global_average_pooling3d[0][0]'
                                                                  ]

 dense_6 (Dense)                (None, 4)            516
['flatten[0][0]']

================================================================================
==================
Total params: 2,397,940
Trainable params: 2,397,940
Non-trainable params: 0
--------------------------------------------------------------------------------
------------------
```

[ ]:
```
model.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),

    optimizer=keras.optimizers.Adam(learning_rate=0.0001),

    metrics=["accuracy"],
)
```

[ ]:
```
history = model.fit(
    x=X_train, y=y_train, epochs=70, validation_data=(X_val,␣
 ↪y_val),callbacks=callbacks
)
```

```
Epoch 1/70
48/48 [==============================] - 237s 4s/step - loss: 1.6361 - accuracy: 0.2418 - val_l
Epoch 2/70
```

```
48/48 [==============================] - 185s 4s/step - loss: 1.4126 - accuracy: 0.2458 - val_l
Epoch 3/70
48/48 [==============================] - 185s 4s/step - loss: 1.4053 - accuracy: 0.2830 - val_l
Epoch 4/70
48/48 [==============================] - 185s 4s/step - loss: 1.4385 - accuracy: 0.2569 - val_l
Epoch 5/70
48/48 [==============================] - 185s 4s/step - loss: 1.3970 - accuracy: 0.2843 - val_l
Epoch 6/70
48/48 [==============================] - 185s 4s/step - loss: 1.3945 - accuracy: 0.2889 - val_l
Epoch 7/70
48/48 [==============================] - 185s 4s/step - loss: 1.4334 - accuracy: 0.2680 - val_l
Epoch 8/70
48/48 [==============================] - 185s 4s/step - loss: 1.4299 - accuracy: 0.2752 - val_l
Epoch 9/70
48/48 [==============================] - 185s 4s/step - loss: 1.3703 - accuracy: 0.3301 - val_l
Epoch 10/70
48/48 [==============================] - 185s 4s/step - loss: 1.3593 - accuracy: 0.3176 - val_l
Epoch 11/70
48/48 [==============================] - 185s 4s/step - loss: 1.3367 - accuracy: 0.3464 - val_l
Epoch 12/70
48/48 [==============================] - 185s 4s/step - loss: 1.2596 - accuracy: 0.4333 - val_l
Epoch 13/70
48/48 [==============================] - 185s 4s/step - loss: 1.1948 - accuracy: 0.4647 - val_l
Epoch 14/70
48/48 [==============================] - 185s 4s/step - loss: 1.0172 - accuracy: 0.5778 - val_l
Epoch 15/70
48/48 [==============================] - 185s 4s/step - loss: 0.8092 - accuracy: 0.6588 - val_l
Epoch 16/70
48/48 [==============================] - 185s 4s/step - loss: 0.6398 - accuracy: 0.7307 - val_l
Epoch 17/70
48/48 [==============================] - 185s 4s/step - loss: 0.4666 - accuracy: 0.8222 - val_l
Epoch 18/70
48/48 [==============================] - 185s 4s/step - loss: 0.3700 - accuracy: 0.8575 - val_l
Epoch 19/70
48/48 [==============================] - 185s 4s/step - loss: 0.2324 - accuracy: 0.9183 - val_l
Epoch 20/70
48/48 [==============================] - 185s 4s/step - loss: 0.1960 - accuracy: 0.9288 - val_l
Epoch 21/70
48/48 [==============================] - 185s 4s/step - loss: 0.1271 - accuracy: 0.9575 - val_l
Epoch 22/70
48/48 [==============================] - 185s 4s/step - loss: 0.0887 - accuracy: 0.9732 - val_l
Epoch 23/70
48/48 [==============================] - 185s 4s/step - loss: 0.0381 - accuracy: 0.9922 - val_l
```

[ ]:
```
model.evaluate(x=X_test, y=y_test, return_dict=True)
```

```
16/16 [==============================] - 24s 2s/step - loss: 0.2508 - accuracy: 0.9061
```

```
{'loss': 0.25083526968955994, 'accuracy': 0.9060665369033813}
```

[ ]:
```
model.save("/kaggle/working/trained_models/cnn_model_35_100x100_v2")
```

2.7.1 Network explanation

For the cnn model we used a 2D+1D convolution model. We found our starting model on tensorflow (https://www.tensorflow.org/tutorials/video/video_classification).

The model exists of 4 main layers each layer is made out of the following:

- conv3D
- layerNormalization
- relu
- conv3D
- layernormalization
- dense layer
- layernormalization

After each layer we halve the video in size. We end the network with global average pooling, flattening and a final dense layer.

With early stopping and restore best weights set to `true` we decided to go from 50 epochs to 70 because sometimes the neural network din't fully converge at epoch 50.

For starters we trained the cnn on on the preprocessed dataset with a batch size of 10 and without early stopping. This gave us an accuracy of around 92% on our trainings and validation set and around 88% on our test set. But on the Kaggle set this model performed rather poorly hitting an accuracy of 62%. Removing the batching (the preprocessed dataset is only 2 gigabytes) and implementing early stopping together with `restore_best_weights=True` gave us an accuracy of 78% on the Kaggle dataset. Now something that helped immensely in the last sprint was increasing the resolution. So we made a dataset with video lenght of 40 frames with a resolution of 200x200px. Putting batch size on 8 because of memory issues we actually got higher accuracies on train, val and test set being around 96% on all three but only reaching the 78.7% mark on Kaggle. In another test we also specified batch size to be 50, which made the model overfit giving us a train accuracy of 97% but a validation accuracy of 92% and a Kaggle accuracy of 62%.

A small error we noticed is that the last dense network has 10 nodes, but we only have 4 categories so changing this to 4 gave us no real improvement.

Next we forgot that not all training classes have the same amount of videos. so we made sure that was the case. We also added block 4 to see if we would be able to extract even more features. It actually gave pretty decent results leaving us with a 93% accuracy but only a 77% on kaggle.

We where playing around with the model a little bit, and changing the last dense layer to have more nodes (40 to be precise) actually drastically improved our results giving us 84% on the Kaggle dataset and 97 percent on the test dataset. Good to mention here is that we also changed the kernel size from the 2 first block to (3,7,7) instead of heaving (3,3,3).

Adding more dense layers and dropout layers actually made the network unable to converge, even a little.

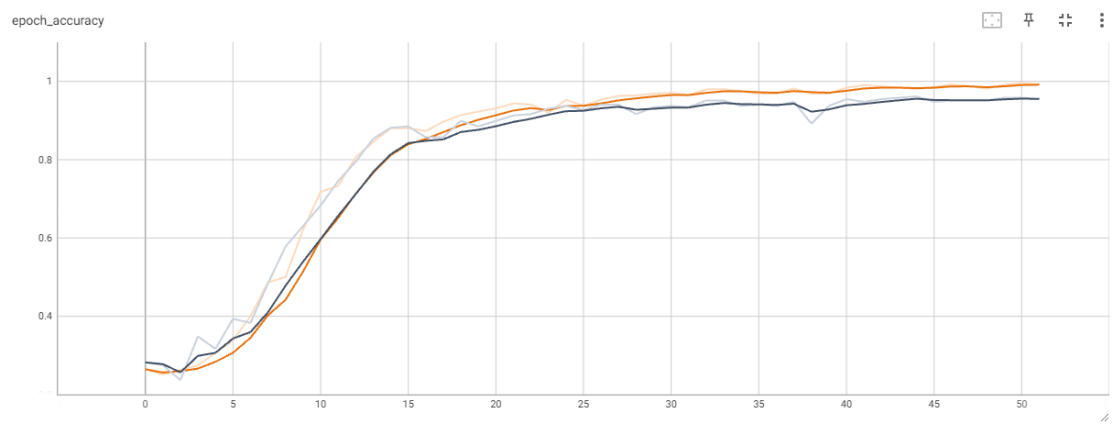
```
x = layers.GlobalAveragePooling3D()(x)
x = layers.Flatten()(x)
x = layers.Dense(4)(x)
x = layers.Dropout(0.8)(x)
x = layers.Dense(4)(x)
x = layers.Dropout(0.8)(x)
x = layers.Dense(4)(x)
x = layers.Dropout(0.8)(x)
x = layers.Dense(4)(x)
```

The network was not able to learn and was stuck at 25% accuracy. even with just the first two dense layers the network refused to learn. So we decided to just go back to one dense layer at the end with a relu activation function. We tried 20, 80 and 50 nodes but nothing gave us as good as a result as 40.

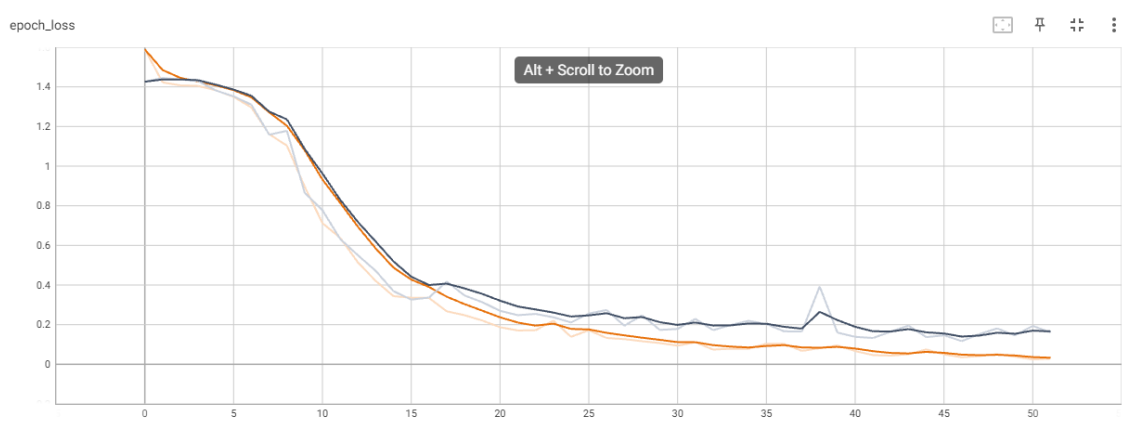
2.7.2 Tensorboard results

We implemented tensorboard to show graphs of the loss value and accuracy changing with each epoch for all training data.

We saved the accuracy per epoch of the neural network: where orange is the accuracy on the trainings set and black the accuracy on the validation set.



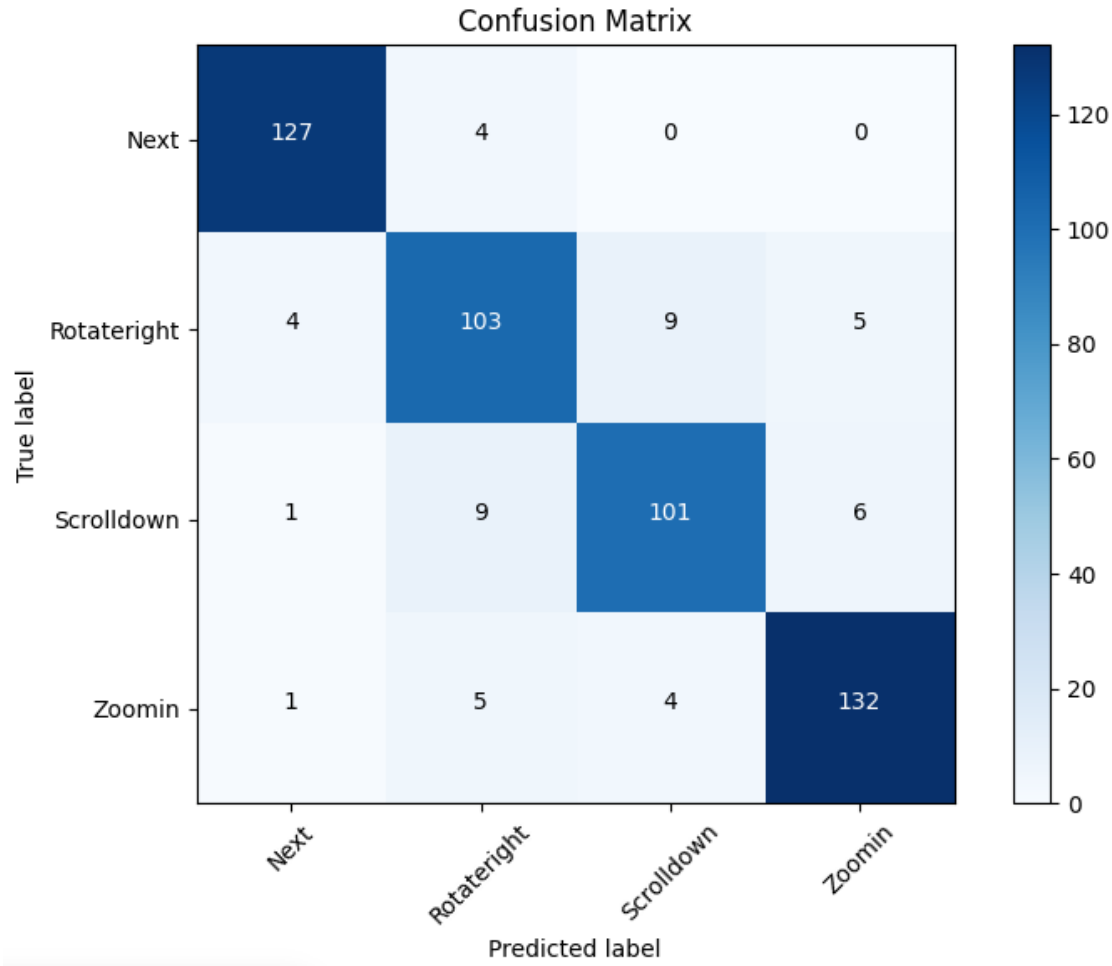
And the loss per epoch of the neural network:



Somethings that stand out is that the accuracy of the validation set sometimes is higher then that of the trainings set. which definitely is a good indication that this network was not overfitting.

2.7.3 Confusion Matrix

[ ]:
```
predictions = get_pred(X_test, model)
cm = confusion_matrix(y_test, predictions, labels=[0, 1, 2, 3])

# Plot confusion matrix with counts
plt.figure(figsize=(8, 6))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()

# Add counts within the cells
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, format(cm[i, j], 'd'),
            horizontalalignment="center",
            color="white" if cm[i, j] > cm.max() / 2 else "black")

tick_marks = np.arange(4)
plt.xticks(tick_marks, list(gesture_labels.keys()), rotation=45)
plt.yticks(tick_marks, list(gesture_labels.keys()))
plt.ylabel('True label')
plt.xlabel('Predicted label')

plt.tight_layout()
plt.show()
```



The confusion matrix predominantly showcases accurate predictions. However, a discernible correlation emerges where the gestures `Rotateright` and `Scrolldown` are occasionally misclassified as each other.

2.8 RNN model

2.8.1 Data (re)processing

Resizing the data In an RNN model, `x` usually has the shape `(batch_size, timesteps, features)`, and `y` `(batch_size, num_classes)`. For our multiclass problem, these correspond to `(video_count, frames_per_video, pixels_per_frame)` and `(video_count, gesture_type)`, with each `gesture_type` of `y` being a one-hot encoded vector.

We can verify whether the current data sizes match the one mentioned above by printing out their shapes:

[28]:
```
print("Training set:")
print(f"\t{X_train.shape=}")
print(f"\t{y_train.shape=}")
```

```
print("\nValidation set:")
print(f"\t{X_val.shape=}")
print(f"\t{y_val.shape=}")

print("\nTest set:")
print(f"\t{X_test.shape=}")
print(f"\t{y_test.shape=}")
```

```
Training set:
        X_train.shape=(1530, 35, 100, 100)
        y_train.shape=(1530,)

Validation set:
        X_val.shape=(511, 35, 100, 100)
        y_val.shape=(511,)

Test set:
        X_test.shape=(511, 35, 100, 100)
        y_test.shape=(511,)
```

The previous shapes clearly do not match those discussed. This can be however easily be solved by using reshaping operations:

[29]:
```
# One-hot encode the labels
y_train_encoded = to_categorical(y_train, num_classes=4)
y_val_encoded = to_categorical(y_val, num_classes=4)
y_test_encoded = to_categorical(y_test, num_classes=4)

# Reshape the input data into 3D tensors
X_train_reshaped = X_train.reshape(X_train.shape[0], X_train.shape[1], -1)
X_val_reshaped = X_val.reshape(X_val.shape[0], X_val.shape[1], -1)
X_test_reshaped = X_test.reshape(X_test.shape[0], X_test.shape[1], -1)

# Print the shapes
print("Training set:")
print(f"\t{X_train_reshaped.shape=}")
print(f"\t{y_train_encoded.shape=}")

print("\nValidation set:")
print(f"\t{X_val_reshaped.shape=}")
print(f"\t{y_val_encoded.shape=}")

print("\nTest set:")
print(f"\t{X_test_reshaped.shape=}")
print(f"\t{y_test_encoded.shape=}")
```

```
Training set:
        X_train_reshaped.shape=(1530, 35, 10000)
```

```
        y_train_encoded.shape=(1530, 4)

Validation set:
        X_val_reshaped.shape=(511, 35, 10000)
        y_val_encoded.shape=(511, 4)

Test set:
        X_test_reshaped.shape=(511, 35, 10000)
        y_test_encoded.shape=(511, 4)
```

Now the data is prepared, a recurrent neural network can be trained.

2.8.2 Prototype RNN

To validate the appropriateness of the newly adjusted input data and to gain a basic understanding of building a recurrent neural network, a prototype will be developed. It's important to note that this initial model's architecture and hyperparameters have partly been created ad random and thus not been optimized. However, there are a few exceptions:

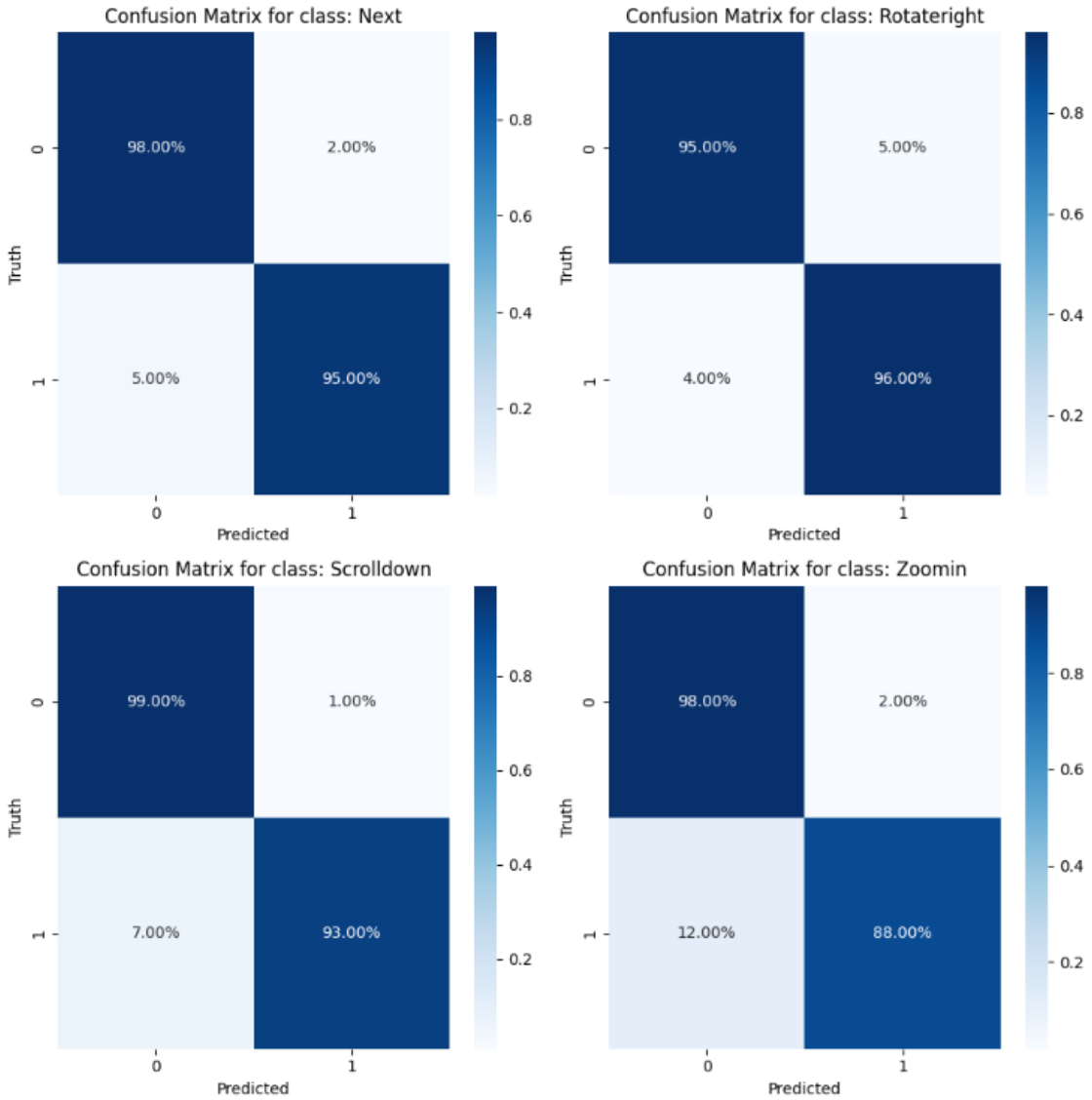
- The **batch size** is rather small, this due to the limitations in available RAM memory when using a (free) Kaggle machine (the higher the batch size, the more data needs to be loaded into the main memory at each given time)
- In this context, the use of a **bidirectional** LSTM, or any other type, is not applicable due to the significance of hand movement direction emphasized in the problem description. For example, while the 'Next' gesture can be executed with either hand, it is consistently performed from left to right.
- In the LSTM layer, the parameter `return_sequences=False` ensures that only the output from the final time step is returned. This effectively transforms the sequence into a single vector, encapsulating the final analysis. Given that there are no subsequent recurrent layers requiring sequential input, this configuration allows the model to focus only on the most relevant information (the last output of the hidden layers) from the sequence for further processing.
- The final output layer has a **size of 4** with a "**softmax**" activation function, this due to our task being a form of "multi-class classification". The model will output a probability for each possible class to which the input video might belong.
- The chosen loss function is "**categorical_crossentropy**" as this is a good choice for (multi-class) classification problems. It penalizes predections that are confident and wrong, which is exactly what we want as each video can only be attributed to a single "class"

[ ]:
```
EPOCHS = 20
BATCH_SIZE = 30

# Create a sequential model
model = keras.Sequential(name="rnn_prototype")

model.add(LSTM(300, input_shape=(X_train_reshaped.shape[1], X_train_reshaped.
 ↪shape[2]), return_sequences=False))
```

```
model.add(layers.Dense(4, activation='softmax'))

# Compile the model & print summary
model.compile(
    optimizer="sgd",
    loss="categorical_crossentropy",
    metrics=[
        "accuracy"
        ]
    )

model.summary()

# Train the model
model.fit(
    X_train_reshaped,
    y_train_encoded,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_reshaped,
        y_val_encoded
        )
    )

print("\nFinal evaluation")
model.evaluate(
    x=X_test_reshaped,
    y=y_test_encoded,
    return_dict=True
    )

# Kaggle only
model.save(f"/kaggle/working/trained_models/rnn_model_35_100x100_{model.name}")
show_confusion_matrix(X_test_reshaped,model)
```

```
Model: "rnn_prototype"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_1 (LSTM)               (None, 300)               12361200

 dense_1 (Dense)             (None, 4)                 1204

=================================================================
Total params: 12362404 (47.16 MB)
Trainable params: 12362404 (47.16 MB)
Non-trainable params: 0 (0.00 Byte)
```

```
----------------------------------------------------------------
Epoch 1/20
53/53 [==============================] - 4s 54ms/step - loss: 1.5081 - accuracy: 0.2498 - val_l
Epoch 2/20
53/53 [==============================] - 2s 32ms/step - loss: 0.4067 - accuracy: 0.9880 - val_l
Epoch 3/20
53/53 [==============================] - 2s 31ms/step - loss: 0.2306 - accuracy: 0.9962 - val_l
Epoch 4/20
53/53 [==============================] - 2s 30ms/step - loss: 0.1657 - accuracy: 0.9968 - val_l
Epoch 5/20
53/53 [==============================] - 2s 31ms/step - loss: 0.1317 - accuracy: 0.9968 - val_l
Epoch 6/20
53/53 [==============================] - 2s 31ms/step - loss: 0.1100 - accuracy: 0.9968 - val_l
Epoch 7/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0947 - accuracy: 0.9981 - val_l
Epoch 8/20
53/53 [==============================] - 2s 31ms/step - loss: 0.0833 - accuracy: 0.9987 - val_l
Epoch 9/20
53/53 [==============================] - 2s 31ms/step - loss: 0.0748 - accuracy: 0.9987 - val_l
Epoch 10/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0679 - accuracy: 0.9987 - val_l
Epoch 11/20
53/53 [==============================] - 2s 31ms/step - loss: 0.0623 - accuracy: 0.9987 - val_l
Epoch 12/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0576 - accuracy: 0.9987 - val_l
Epoch 13/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0536 - accuracy: 0.9987 - val_l
Epoch 14/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0501 - accuracy: 0.9987 - val_l
Epoch 15/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0471 - accuracy: 0.9987 - val_l
Epoch 16/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0443 - accuracy: 0.9987 - val_l
Epoch 17/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0419 - accuracy: 0.9987 - val_l
Epoch 18/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0398 - accuracy: 0.9994 - val_l
Epoch 19/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0380 - accuracy: 0.9994 - val_l
Epoch 20/20
53/53 [==============================] - 2s 30ms/step - loss: 0.0363 - accuracy: 0.9994 - val_l

Final evaluation
17/17 [==============================] - 0s 18ms/step - loss: 1.5381 - accuracy: 0.2841
```



Conclusion When looking at the above results, it is clear that the model quickly learned to perfectly predict the training data, achieving an accuracy of 99.94% by the second epoch. However, this did not translate to good performance on the validation or test data, with the accuracy remaining around 25% for those. The latter can also be observed in the confusion matrices, where a positive prediction is (roughly) equally given for both actual negative and actual positive instances, meaning a positive prediction is in no way correlated to the true gesture type. This indicates the model is simply guessing when presented with new data, which is not the desired behavior.

Another observation that can be made is that the model finished all 20 epochs during training even though the accuracy did not change after the second epoch. This is due to the absence of callback functions (as this is a mere prototype). A useful addition would thus be the implementation of early stopping, e.g., once the model's accuracy reaches a certain threshold or does not increase significantly within a predefined number of epochs.

Since this simple model does not implement any regularization and is already majorly overfitting on the entire dataset, the decision was made not to tune any of its hyperparameters (e.g., learning

rate) on a subset of the data. Instead, the focus will be shifted towards improving the model's architecture and implementing the previously discussed techniques up until an actual 'sane' output is reliably generated.

2.8.3 RNN model V1 - Introducing data scaling

The current hypothesis is that despite the input data's size meeting the requirements, the actual feature values are scaled too broadly, encompassing both positive and negative values greater than (absolute) 1. This wide range could hinder the model's ability to extract useful information. To address this issue, The input data for this model will be rescaled to a range between 0 and 1 using a `MinMaxScaler`.

[30]:
```
scaler = MinMaxScaler()

# Reshape for scaling
X_train_reshaped = X_train.reshape(X_train.shape[0]*X_train.shape[1], -1)
X_val_reshaped = X_val.reshape(X_val.shape[0]*X_val.shape[1], -1)
X_test_reshaped = X_test.reshape(X_test.shape[0]*X_test.shape[1], -1)

# Fit and transform
X_train_scaled = scaler.fit_transform(X_train_reshaped)
X_val_scaled = scaler.transform(X_val_reshaped)
X_test_scaled = scaler.transform(X_test_reshaped)

# Reshape back to original shape
X_train_scaled = X_train_scaled.reshape(X_train.shape[0], X_train.shape[1], -1)
X_val_scaled = X_val_scaled.reshape(X_val.shape[0], X_val.shape[1], -1)
X_test_scaled = X_test_scaled.reshape(X_test.shape[0], X_test.shape[1], -1)

# Print the shapes
print("Training set:")
print(f"\t{X_train_scaled.shape=}")
print(f"\t{y_train_encoded.shape=}")

print("\nValidation set:")
print(f"\t{X_val_scaled.shape=}")
print(f"\t{y_val_encoded.shape=}")

print("\nTest set:")
print(f"\t{X_test_scaled.shape=}")
print(f"\t{y_test_encoded.shape=}")
```

```
Training set:
        X_train_scaled.shape=(1530, 35, 10000)
        y_train_encoded.shape=(1530, 4)

Validation set:
        X_val_scaled.shape=(511, 35, 10000)
```

```
        y_val_encoded.shape=(511, 4)

Test set:
        X_test_scaled.shape=(511, 35, 10000)
        y_test_encoded.shape=(511, 4)
```

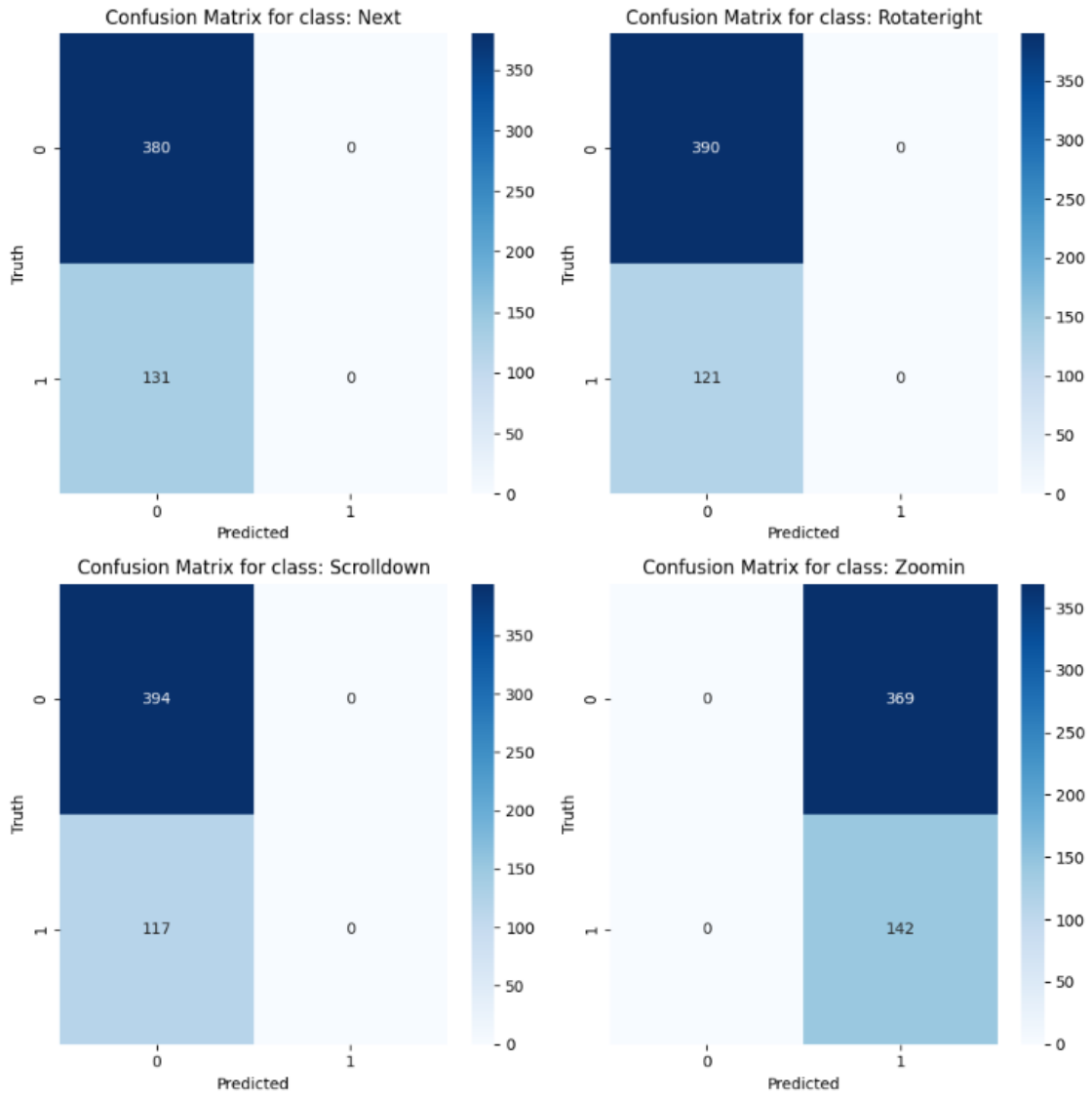
To solve the issue of the model continueing training even when the training accuracy has reached 100%, callbacks have been introduced to this model. These include early stopping during training when either an accuracy of .99 has been reached or when the accuracy stabilizes for a set count of epochs.

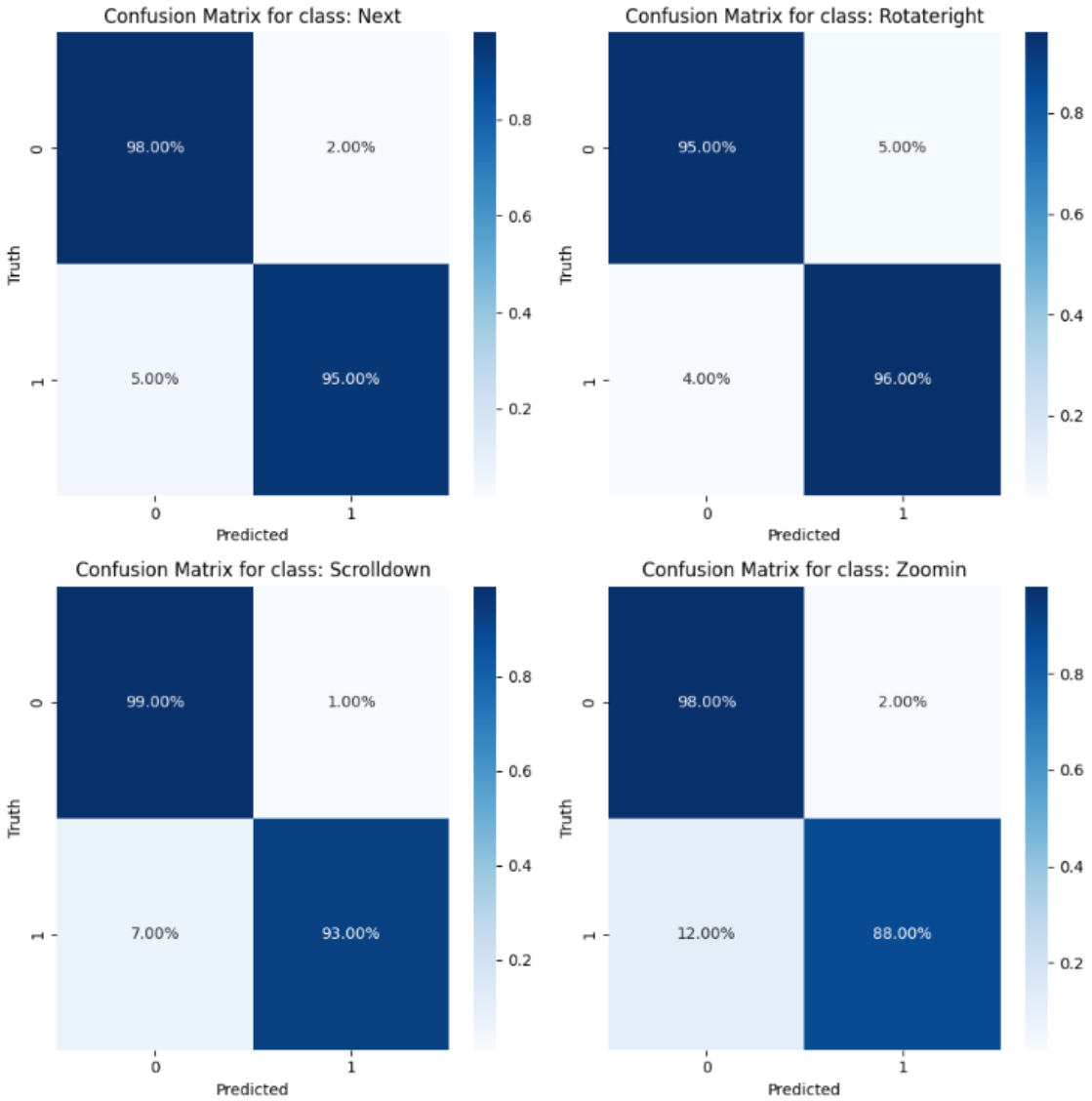
[ ]:
```
EPOCHS = 20
BATCH_SIZE = 30

# Create a sequential model
model = keras.Sequential(name="rnn_V1")

model.add(LSTM(64, input_shape=(X_train_scaled.shape[1], X_train_scaled.
 ↪shape[2]), return_sequences=False))
model.add(layers.Dense(4, activation='softmax'))

# Compile the model & print summary
model.compile(
    optimizer="sgd",
    loss="categorical_crossentropy",
    metrics=[
        "accuracy"
        ]
    )

model.summary()

# Train the model
model.fit(
    X_train_scaled,
    y_train_encoded,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_scaled,
        y_val_encoded
        ),
    callbacks=callbacks
    )

print("\nFinal evaluation")
model.evaluate(
    x=X_test_scaled,
```

```
    y=y_test_encoded,
    return_dict=True
    )

# Kaggle only
model.save(f"/kaggle/working/trained_models/rnn_model_35_100x100_{model.name}")
show_confusion_matrix(X_test_scaled,model, relative=True)
show_confusion_matrix(X_test_scaled,model, relative=False)
```

```
Model: "rnn_V1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_20 (LSTM)              (None, 300)               12361200

 dense_10 (Dense)            (None, 4)                 1204

=================================================================
Total params: 12362404 (47.16 MB)
Trainable params: 12362404 (47.16 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/20
51/51 [==============================] - 8s 125ms/step - loss: 1.4587 - accuracy: 0.2562 - val_
Epoch 2/20
51/51 [==============================] - 4s 75ms/step - loss: 1.4172 - accuracy: 0.2490 - val_
Epoch 3/20
51/51 [==============================] - 4s 72ms/step - loss: 1.4057 - accuracy: 0.2523 - val_
Epoch 4/20
51/51 [==============================] - 4s 75ms/step - loss: 1.4028 - accuracy: 0.2484 - val_
Epoch 5/20
51/51 [==============================] - 4s 72ms/step - loss: 1.4026 - accuracy: 0.2373 - val_
Epoch 6/20
51/51 [==============================] - 4s 77ms/step - loss: 1.3994 - accuracy: 0.2614 - val_
Epoch 7/20
51/51 [==============================] - 4s 74ms/step - loss: 1.4030 - accuracy: 0.2477 - val_
Epoch 8/20
51/51 [==============================] - 4s 73ms/step - loss: 1.3998 - accuracy: 0.2484 - val_
Epoch 9/20
51/51 [==============================] - 4s 75ms/step - loss: 1.3965 - accuracy: 0.2614 - val_
Epoch 10/20
51/51 [==============================] - 4s 73ms/step - loss: 1.3964 - accuracy: 0.2444 - val_
Epoch 11/20
51/51 [==============================] - 4s 74ms/step - loss: 1.3968 - accuracy: 0.2529 - val_
Epoch 12/20
51/51 [==============================] - 4s 75ms/step - loss: 1.3939 - accuracy: 0.2699 - val_
Epoch 13/20
51/51 [==============================] - 4s 74ms/step - loss: 1.3942 - accuracy: 0.2510 - val_
```

```
Epoch 14/20
51/51 [==============================] - 4s 73ms/step - loss: 1.3972 - accuracy: 0.2399 - val_l
Epoch 15/20
51/51 [==============================] - 4s 76ms/step - loss: 1.3943 - accuracy: 0.2458 - val_l
Epoch 16/20
51/51 [==============================] - 4s 74ms/step - loss: 1.3942 - accuracy: 0.2660 - val_l
Epoch 17/20
51/51 [==============================] - 4s 73ms/step - loss: 1.3958 - accuracy: 0.2569 - val_l
Epoch 18/20
51/51 [==============================] - 4s 73ms/step - loss: 1.3942 - accuracy: 0.2497 - val_l

Final evaluation
16/16 [==============================] - 0s 24ms/step - loss: 1.3845 - accuracy: 0.2779
```





Conclusion This model, in contrast to the prototype, does not overfit the training data. Instead, it exhibits a consistent lack of learning, as evidenced by the stagnant accuracy and loss values across multiple epochs. It appears that the model is unable to extract meaningful patterns from the data, resulting in what seems to be random guessing. The final evaluation on the test set also confirmed this, with an accuracy of again (close to) 25%.

The confusion matrices provide a clear insight , as they indicate that the model consistently predicts the same gesture. This consistency accounts for the 25% probability of a correct prediction, as the input data is uniformly distributed over the 4 gesture types.

The formulated hypothesis consequently is either completely false or has to be modified. Consequently, as adding complexity to the model architecture is often an easy way to fix underfitting, this is exaclty what will be tried next.

2.8.4 RNN model V2 - Adding complexity

To increase the previous model's complexity, this iteration contains additional LSTM layers in the architecture. Specifically, it now includes three LSTM layers with 900 units each, as opposed to the single LSTM layer in the previous model. The anticipated outcome of this added "depth" is an improved ability for the model to learn more complex patterns within the data, thereby solving the current issue of underfitting.

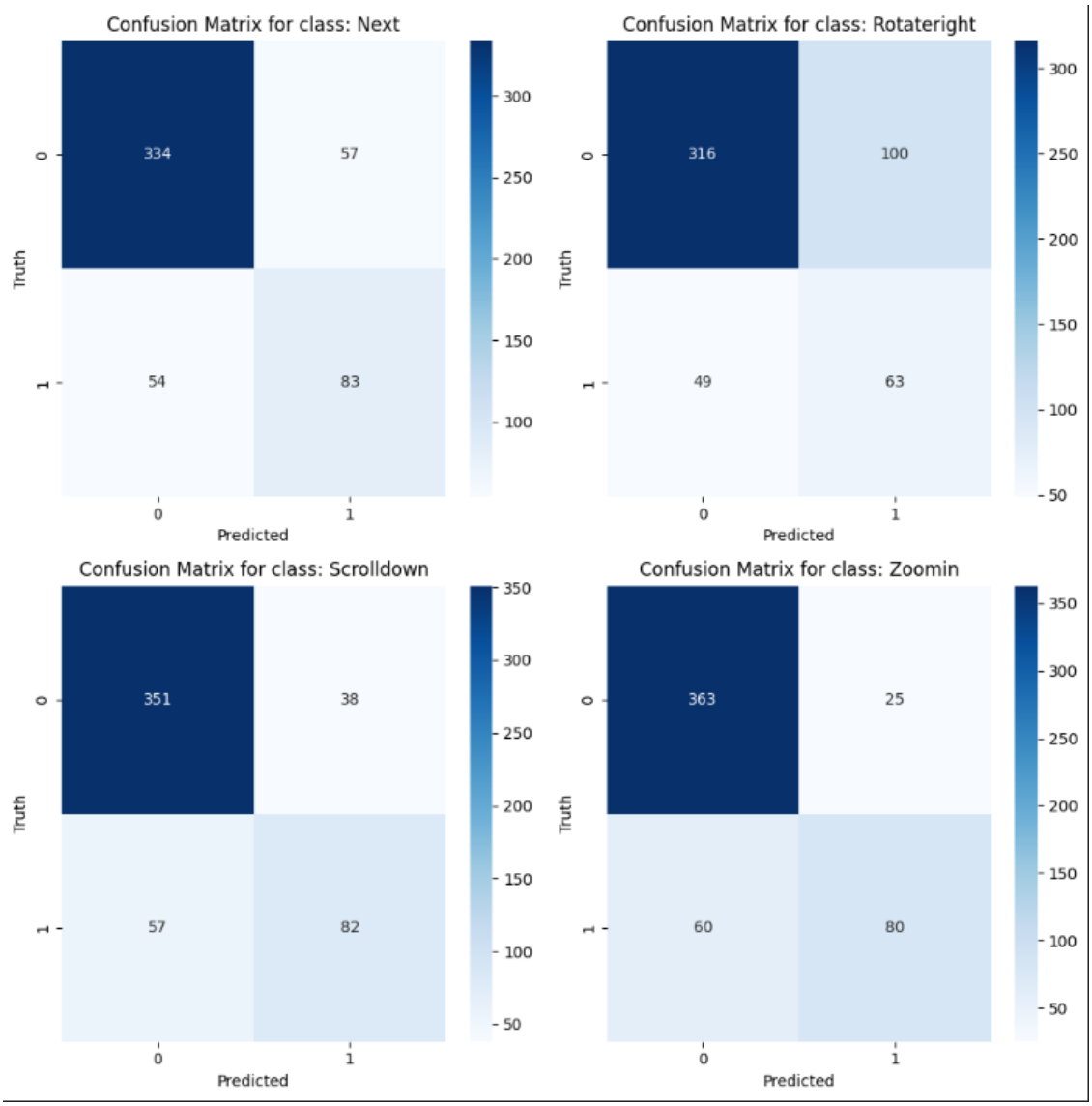
To provide a more comprehensive view of the model's performance, `Precision`,`CategoricalCrossentropy` and `Recall` have also been added as metrics.

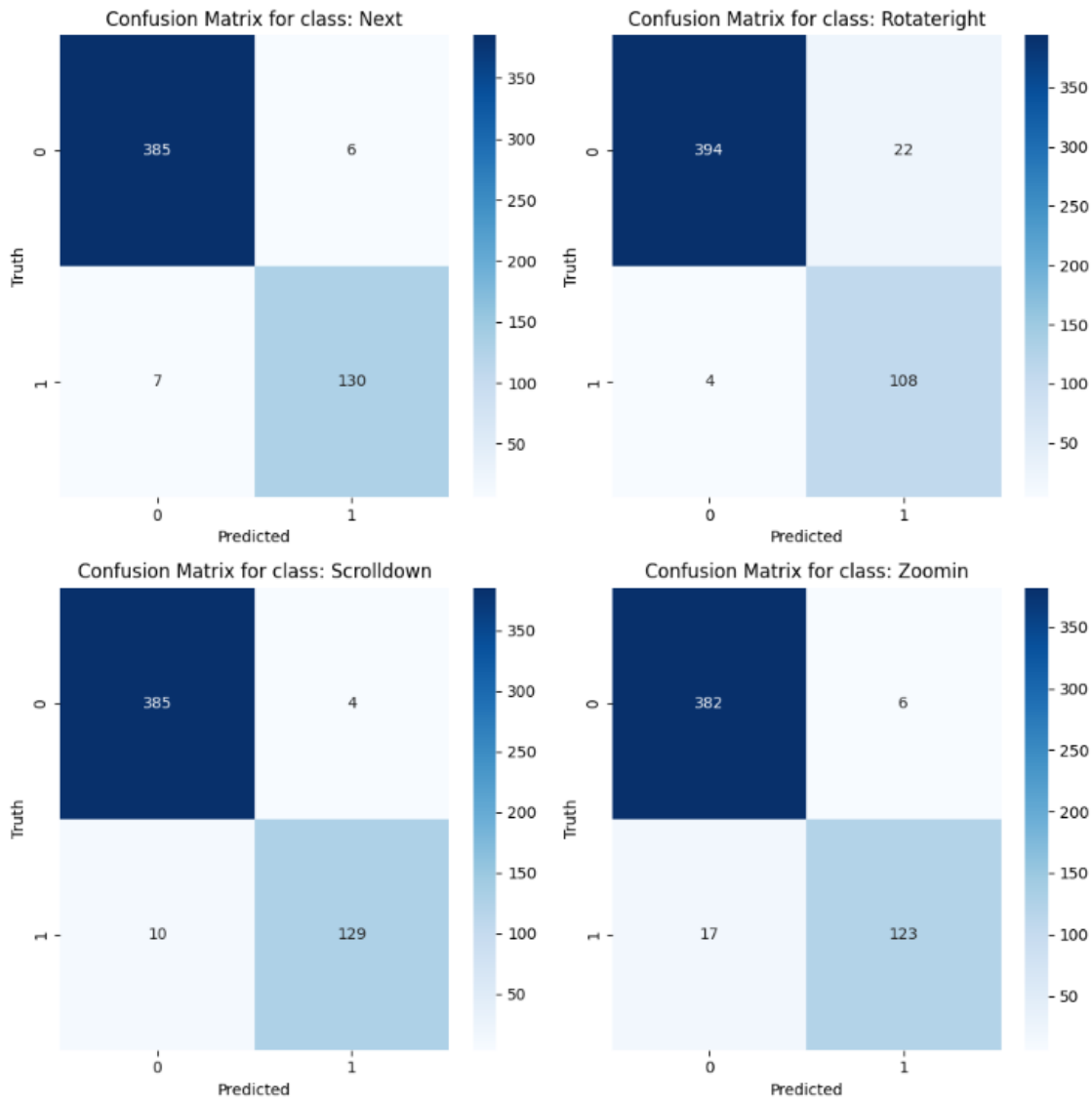
[ ]:
```
EPOCHS = 20
BATCH_SIZE = 30

# Create a sequential model
model = keras.Sequential(name="rnn_V2.1")

model.add(LSTM(512, input_shape=(X_train_scaled.shape[1], X_train_scaled.
 ↪shape[2]), return_sequences=True))
model.add(layers.LSTM(512,return_sequences=True))
model.add(layers.LSTM(512,return_sequences=False))
model.add(layers.Dense(4, activation='softmax'))

# Compile the model
model.compile(
    optimizer="sgd",
    loss="categorical_crossentropy",
    metrics=[
        "accuracy",
        metrics.Precision(),
        metrics.CategoricalCrossentropy() ,
        metrics.Recall(),
        ]
)
model.summary()

# Train the model
model.fit(
    X_train_scaled,
    y_train_encoded,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_scaled,
        y_val_encoded
        ),
    callbacks=callbacks
)
```

```python
print("\nFinal evaluation")
model.evaluate(
    x=X_test_scaled,
    y=y_test_encoded,
    return_dict=True
)

# Kaggle only
model.save(f"/kaggle/working/trained_models/rnn_model_35_100x100_{model.name}")
show_confusion_matrix(X_test_scaled,model, relative=True)
show_confusion_matrix(X_test_scaled,model, relative=False)
```

```
Model: "rnn_V2.1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_15 (LSTM)              (None, 35, 512)           21530624

 lstm_16 (LSTM)              (None, 35, 512)           2099200

 lstm_17 (LSTM)              (None, 512)               2099200

 dense_5 (Dense)             (None, 4)                 2052

=================================================================
Total params: 25731076 (98.16 MB)
Trainable params: 25731076 (98.16 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/20
53/53 [==============================] - 15s 198ms/step - loss: 1.4054 - accuracy: 0.2397 - pre
Epoch 2/20
53/53 [==============================] - 7s 131ms/step - loss: 1.3956 - accuracy: 0.2612 - prec
Epoch 3/20
53/53 [==============================] - 7s 134ms/step - loss: 1.3991 - accuracy: 0.2498 - prec
Epoch 4/20
53/53 [==============================] - 7s 133ms/step - loss: 1.3976 - accuracy: 0.2372 - prec
Epoch 5/20
53/53 [==============================] - 7s 132ms/step - loss: 1.3945 - accuracy: 0.2410 - prec
Epoch 6/20
53/53 [==============================] - 7s 137ms/step - loss: 1.3962 - accuracy: 0.2359 - prec
Epoch 7/20
53/53 [==============================] - 7s 131ms/step - loss: 1.3933 - accuracy: 0.2460 - prec
Epoch 8/20
53/53 [==============================] - 7s 131ms/step - loss: 1.3931 - accuracy: 0.2467 - prec
Epoch 9/20
53/53 [==============================] - 7s 131ms/step - loss: 1.3930 - accuracy: 0.2454 - prec
```

```
Epoch 10/20
53/53 [==============================] - 7s 132ms/step - loss: 1.3937 - accuracy: 0.2486 - prec

Final evaluation
17/17 [==============================] - 1s 34ms/step - loss: 1.3887 - accuracy: 0.2652 - preci
```





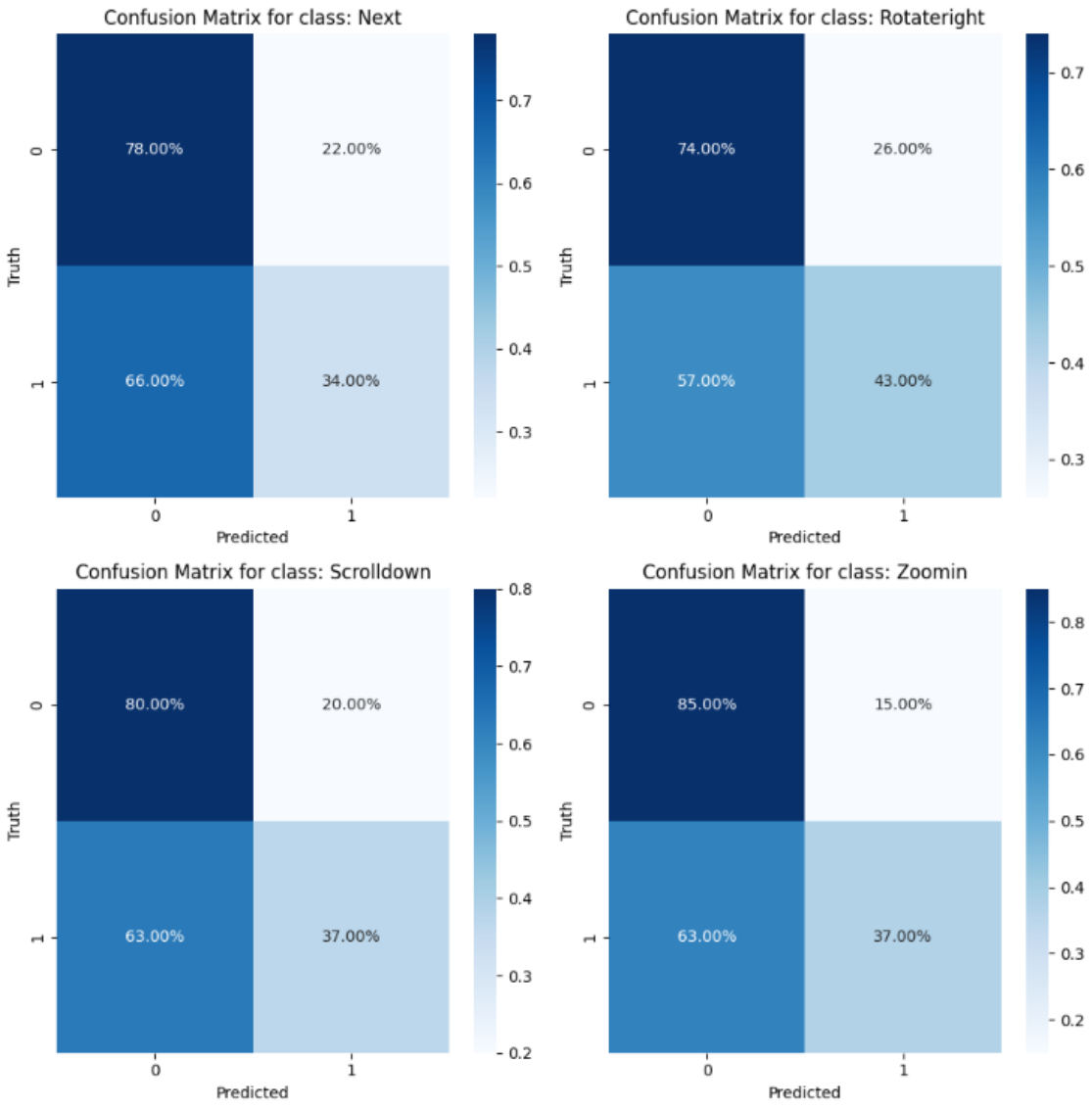
Given that the performance of this model mirrors that of the previous one, it appears that the data scaling approach employed may be at fault. To confirm this, the initial (non-rescaled) data will be fed to the same model:

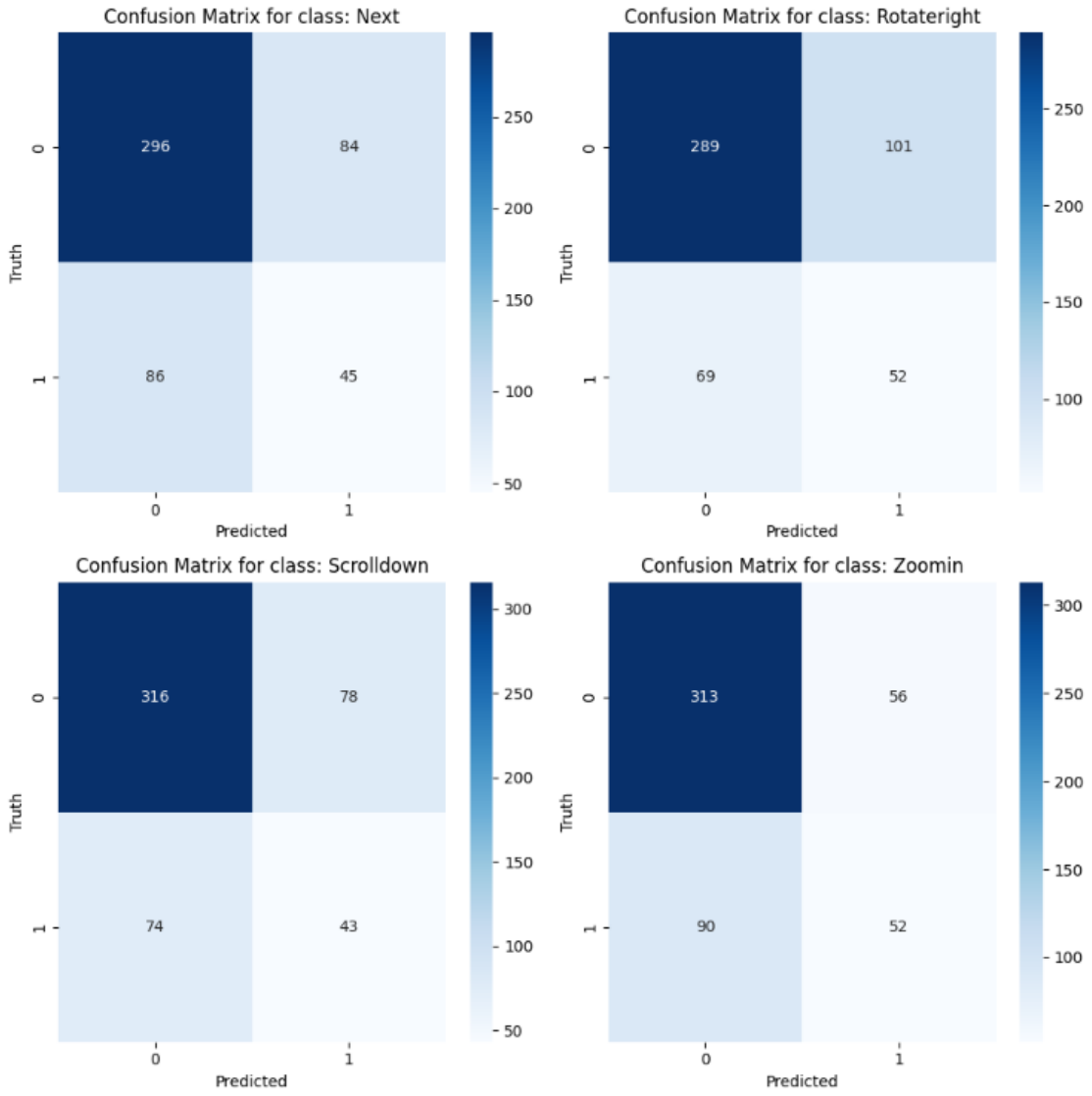
[ ]:
```
EPOCHS = 30
BATCH_SIZE = 20

# Create a sequential model
model = keras.Sequential(name="rnn_v2.2")

model.add(layers.LSTM(900, input_shape=(X_train_reshaped.shape[1],␣
 ↪X_train_reshaped.shape[2]), return_sequences=True))
model.add(layers.LSTM(900,return_sequences=True))
model.add(layers.LSTM(900,return_sequences=False))
model.add(layers.Dense(4, activation='softmax'))
```

```python
# Compile the model
model.compile(
    optimizer="sgd",
    loss="categorical_crossentropy",
    metrics=[
        "accuracy",
        metrics.Precision(),
        metrics.CategoricalCrossentropy() ,
        metrics.Recall(),
        ]
)

model.summary()

# Train the model
model.fit(
    X_train_reshaped,
    y_train_encoded,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_reshaped,
        y_val_encoded
        ),
    callbacks=callbacks
)

print("\nFinal evaluation")
model.evaluate(
    x=X_test_reshaped,
    y=y_test_encoded,
    return_dict=True
)

# Kaggle only
model.save(f"/kaggle/working/trained_models/rnn_model_35_100x100_{model.name}")
show_confusion_matrix(X_test_reshaped,model, relative=True)
show_confusion_matrix(X_test_reshaped,model, relative=False)
```

```
Model: "rnn_v2.2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_14 (LSTM)              (None, 35, 900)           39243600

 lstm_15 (LSTM)              (None, 35, 900)           6483600
```

```
 lstm_16 (LSTM)              (None, 900)               6483600

 dense_6 (Dense)             (None, 4)                 3604

=================================================================
Total params: 52214404 (199.18 MB)
Trainable params: 52214404 (199.18 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/30
102/102 [==============================] - 20s 159ms/step - loss: 1.3803 - accuracy: 0.2856 - p
Epoch 2/30
102/102 [==============================] - 15s 143ms/step - loss: 0.9935 - accuracy: 0.9935 - p

Final evaluation
16/16 [==============================] - 1s 38ms/step - loss: 1.3284 - accuracy: 0.3757 - preci
```


Confusion Matrix for class: Next
78.00%
22.00%
66.00%
34.00%
Truth
Predicted
Confusion Matrix for class: Rotateright
74.00%
26.00%
57.00%
43.00%
Truth
Predicted
Confusion Matrix for class: Scrolldown
80.00%
20.00%
63.00%
37.00%
Truth
Predicted
Confusion Matrix for class: Zoomin
85.00%
15.00%
63.00%
37.00%
Truth
Predicted



Conclusion The increase of architectural complexity did not yield any improvements in the model's behavior when trained on the scaled data. This outcome proves that the (input regularization) hypothesis was incorrect, hence rendering the use of `MinMaxScaler` ineffective in this context.

However, when the model was trained on non-scaled data, it achieved an unexpected accuracy of 38% on the test set. Further examination of the confusion matrix revealed an average true positive rate of approximately 35%, with the "Rotate right" gesture reaching as high as 43%. These figures are too substantial to be dismissed as mere anomalies, suggesting that the model has indeed learned something.

Although overfitting remains a significant issue, now even more than before (given it only took 2 epochs to reach near perfect accuracy during training), the current model did show promising results. If this obstacle can be overcome by using regularization techniques or implementing dropout, a usable accuracy might be achieved.

2.8.5 RNN model V3 - Limit overfitting

This iteration introduces three main changes to the model: the addition of L2 regularization to the first layer, the introduction of dropout, and a decrease in the learning rate to 0.001 (from the default 0.01).

The L2 regularization encourages the model to use smaller weights, leading to a simpler model that is less prone to overfitting. The introduction of dropout further prevents the model from memorizing the training data, thereby enhancing its generalization capability. Lastly, by lowering the learning rate, the model's training becomes more stable and controlled, allowing for more precise adjustments to the model's weights during the learning process.

[ ]:
```
EPOCHS = 150
BATCH_SIZE = 5

# Create a sequential model
model = keras.Sequential(name="rnn_v3")

model.add(layers.LSTM(900, return_sequences=True, kernel_regularizer=l2(0.01),␣
 ↪input_shape=(X_train_reshaped.shape[1], X_train_reshaped.shape[2])))
model.add(layers.LSTM(900, return_sequences=True, dropout=0.1))
model.add(layers.LSTM(900, return_sequences=False, dropout=0.1))
model.add(layers.Dense(4, activation='softmax'))

model.build((X_train_reshaped.shape[1], X_train_reshaped.shape[2]))

# Compile the model
model.compile(
    optimizer=optimizers.SGD(learning_rate=0.0001),
    loss="categorical_crossentropy",
    metrics=[
        "accuracy",
        metrics.Precision(),
        metrics.CategoricalCrossentropy()
        ]
    )

model.summary()

# Train the model
model.fit(
    X_train_reshaped,
    y_train_encoded,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_reshaped,
        y_val_encoded
```

```
        ),
    callbacks=callbacks
    )

print("\nFinal evaluation")
model.evaluate(
    x=X_test_reshaped,
    y=y_test_encoded,
    return_dict=True
    )

# Kaggle only
model.save(f"/kaggle/working/trained_models/rnn_model_35_100x100_{model.name}")
show_confusion_matrix(X_test_reshaped,model)
```

The training of this model, anticipated to be time-consuming, was left unattended. Due to an oversight, the predefined callback functions were not correctly implemented and the training process subsequently continued uninterrupted, despite achieving near-perfect accuracy within just 2 to 3 epochs. This led to the model accidentally learning from both the validation and training sets. Surprisingly, this model achieved a 75% accuracy on the test set, suggesting that it was indeed learning and not merely making random guesses when faced with new data.

Given that the training and validation data constitute 80% of the available local data, one possible explanation for the model's performance could be its ability to match the "new" data with patterns it had already learned during training. This suggests that the model may not be generalizing well to completely unseen data, but rather recognizing and recalling patterns from the extensive training data. To confirm this hypothesis and ensure the model's robustness, the Kaggle competition dataset was utilized as an independent test set.

With this completely new data, the model achieved an accuracy of 54%. This means that while the model is indeed overfitting, as evidenced by the 20% drop in accuracy compared to the local testing data, its performance is still reasonably good.

This suggests that the model's performance could potentially be enhanced by feeding it more data. However, since attaining the highest possible accuracy is not the primary objective of this project, no additional time will be spent on this "exploit".

```
Model: "rnn_v3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_27 (LSTM)              (None, 35, 900)           39243600

 lstm_28 (LSTM)              (None, 35, 900)           6483600

 lstm_29 (LSTM)              (None, 900)               6483600

 dense_8 (Dense)             (None, 4)                 3604

=================================================================
```

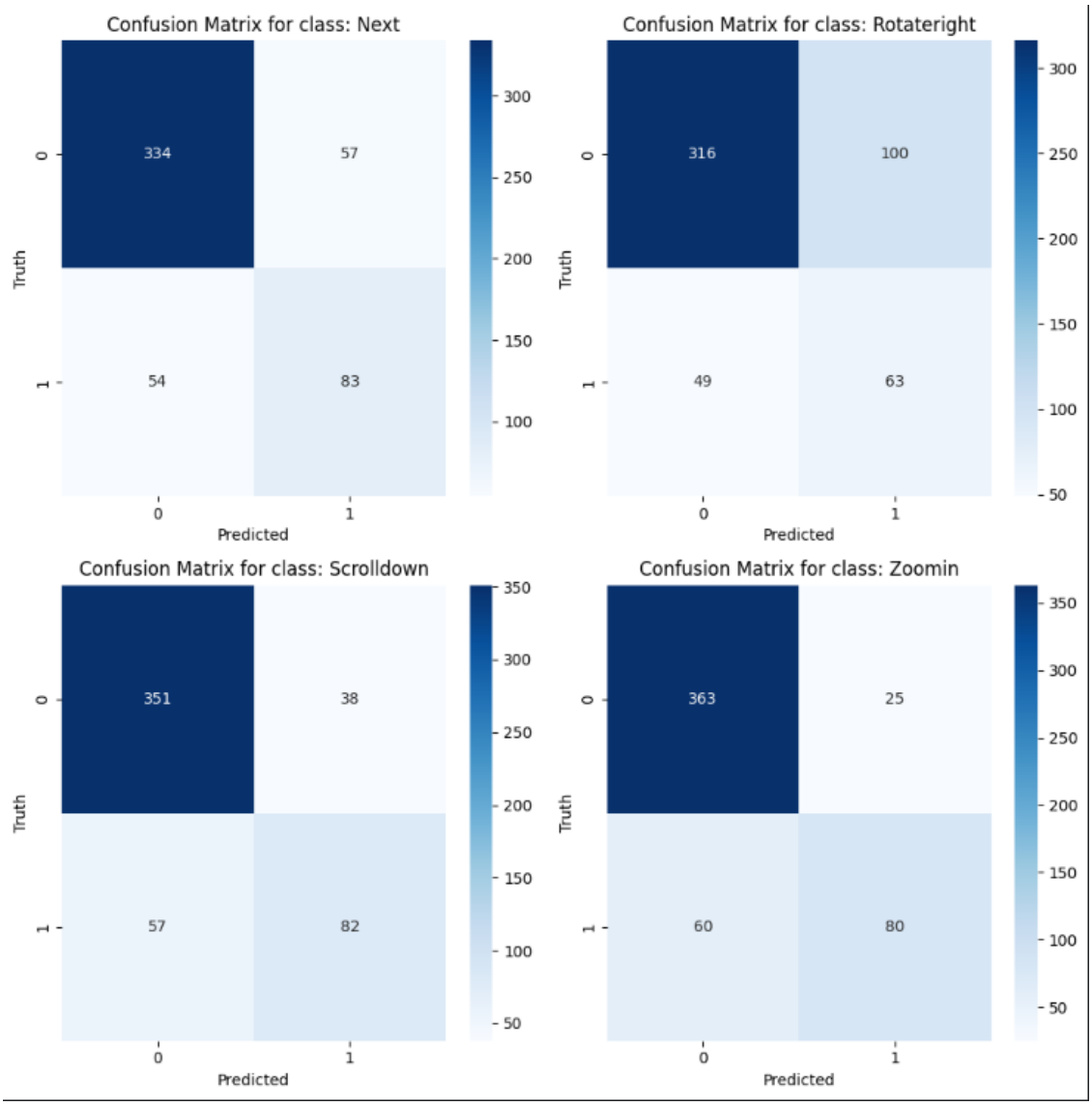
```
Total params: 52214404 (199.18 MB)
Trainable params: 52214404 (199.18 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/150
106/106 [==============================] - 22s 159ms/step - loss: 49.2997 - accuracy: 0.3757 -
Epoch 2/150
106/106 [==============================] - 15s 138ms/step - loss: 40.0197 - accuracy: 0.9798 -
Epoch 3/150
106/106 [==============================] - 15s 139ms/step - loss: 33.3778 - accuracy: 0.9987 -
Epoch 4/150
106/106 [==============================] - 15s 138ms/step - loss: 27.0509 - accuracy: 1.0000 -
Epoch 5/150
106/106 [==============================] - 15s 141ms/step - loss: 21.8808 - accuracy: 1.0000 -
Epoch 6/150
106/106 [==============================] - 15s 139ms/step - loss: 17.6990 - accuracy: 1.0000 -
Epoch 7/150
106/106 [==============================] - 15s 142ms/step - loss: 14.3164 - accuracy: 1.0000 -
Epoch 8/150
106/106 [==============================] - 14s 137ms/step - loss: 11.5803 - accuracy: 1.0000 -
Epoch 9/150
106/106 [==============================] - 15s 141ms/step - loss: 9.3671 - accuracy: 1.0000 - p
Epoch 10/150
106/106 [==============================] - 15s 138ms/step - loss: 7.5769 - accuracy: 1.0000 - p
Epoch 11/150
106/106 [==============================] - 15s 139ms/step - loss: 6.1289 - accuracy: 1.0000 - p
Epoch 12/150
106/106 [==============================] - 15s 138ms/step - loss: 4.9576 - accuracy: 1.0000 - p
Epoch 13/150
106/106 [==============================] - 15s 138ms/step - loss: 4.0102 - accuracy: 1.0000 - p
Epoch 14/150
106/106 [==============================] - 15s 138ms/step - loss: 3.2438 - accuracy: 1.0000 - p
Epoch 15/150
106/106 [==============================] - 15s 138ms/step - loss: 2.6239 - accuracy: 1.0000 - p
Epoch 16/150
106/106 [==============================] - 15s 139ms/step - loss: 2.1225 - accuracy: 1.0000 - p
Epoch 17/150
106/106 [==============================] - 15s 138ms/step - loss: 1.7169 - accuracy: 1.0000 - p
Epoch 18/150
106/106 [==============================] - 15s 142ms/step - loss: 1.3888 - accuracy: 1.0000 - p
Epoch 19/150
106/106 [==============================] - 14s 137ms/step - loss: 1.1235 - accuracy: 1.0000 - p
Epoch 20/150
106/106 [==============================] - 15s 140ms/step - loss: 0.9088 - accuracy: 1.0000 - p
Epoch 21/150
106/106 [==============================] - 14s 137ms/step - loss: 0.7352 - accuracy: 1.0000 - p
Epoch 22/150
106/106 [==============================] - 15s 140ms/step - loss: 0.5947 - accuracy: 1.0000 - p
```

```
Epoch 23/150
106/106 [==============================] - 15s 139ms/step - loss: 0.4811 - accuracy: 1.0000 - p
Epoch 24/150
106/106 [==============================] - 14s 137ms/step - loss: 0.3893 - accuracy: 1.0000 - p
Epoch 25/150
106/106 [==============================] - 14s 136ms/step - loss: 0.3149 - accuracy: 1.0000 - p
Epoch 26/150
106/106 [==============================] - 15s 140ms/step - loss: 0.2548 - accuracy: 1.0000 - p
Epoch 27/150
106/106 [==============================] - 15s 140ms/step - loss: 0.2062 - accuracy: 1.0000 - p
Epoch 28/150
106/106 [==============================] - 15s 139ms/step - loss: 0.1668 - accuracy: 1.0000 - p
Epoch 29/150
106/106 [==============================] - 15s 139ms/step - loss: 0.1350 - accuracy: 1.0000 - p
Epoch 30/150
106/106 [==============================] - 15s 139ms/step - loss: 0.1093 - accuracy: 1.0000 - p
Epoch 31/150
106/106 [==============================] - 15s 143ms/step - loss: 0.0885 - accuracy: 1.0000 - p
Epoch 32/150
106/106 [==============================] - 15s 139ms/step - loss: 0.0716 - accuracy: 1.0000 - p
Epoch 33/150
106/106 [==============================] - 15s 138ms/step - loss: 0.0580 - accuracy: 1.0000 - p
Epoch 34/150
106/106 [==============================] - 14s 138ms/step - loss: 1.2675 - accuracy: 0.9096 - p
Epoch 35/150
106/106 [==============================] - 15s 139ms/step - loss: 8.3052 - accuracy: 0.6667 - p
Epoch 36/150
106/106 [==============================] - 14s 135ms/step - loss: 8.0267 - accuracy: 0.9968 - p
Epoch 37/150
106/106 [==============================] - 15s 139ms/step - loss: 6.5491 - accuracy: 1.0000 - p
Epoch 38/150
106/106 [==============================] - 14s 137ms/step - loss: 5.2972 - accuracy: 1.0000 - p
Epoch 39/150
106/106 [==============================] - 15s 139ms/step - loss: 4.2848 - accuracy: 1.0000 - p

Final evaluation
17/17 [==============================] - 1s 39ms/step - loss: 0.8156 - accuracy: 0.7557 - preci
```

After implementing the callback functions correctly, the model's results are as follows:

```
Model: "rnn_v3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_26 (LSTM)              (None, 35, 900)           39243600

 lstm_27 (LSTM)              (None, 35, 900)           6483600

 lstm_28 (LSTM)              (None, 900)               6483600
```

```
 dense_12 (Dense)            (None, 4)                 3604

=================================================================
Total params: 52214404 (199.18 MB)
Trainable params: 52214404 (199.18 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/150
153/153 [==============================] - 31s 176ms/step - loss: nan - accuracy: 0.2477 - prec
Epoch 2/150
153/153 [==============================] - 24s 158ms/step - loss: nan - accuracy: 0.2477 - prec
Epoch 3/150
153/153 [==============================] - 24s 159ms/step - loss: nan - accuracy: 0.2477 - prec
Epoch 4/150
153/153 [==============================] - 25s 162ms/step - loss: nan - accuracy: 0.2477 - prec
Epoch 5/150
153/153 [==============================] - 24s 160ms/step - loss: nan - accuracy: 0.2477 - prec

Final evaluation
16/16 [==============================] - 1s 45ms/step - loss: nan - accuracy: 0.2564 - precisio
16/16 [==============================] - 1s 39ms/step
```



Upon retraining, the model unexpectedly reproduced the same issue as before, consistently predicting the same gesture. To investigate whether the model was defaulting to predict "Next", multiple reruns were executed. In each instance, the model appeared to arbitrarily select a gesture and then consistently predict that gesture.

Upon further inspection of the confusion matrices, it was found that during the division of the dataset into training, validation, and test subsets, the distribution of gestures was not always uniform. This uneven distribution could potentially lead to the overrepresentation of a particular gesture during training, leading to the model selecting said gesture.

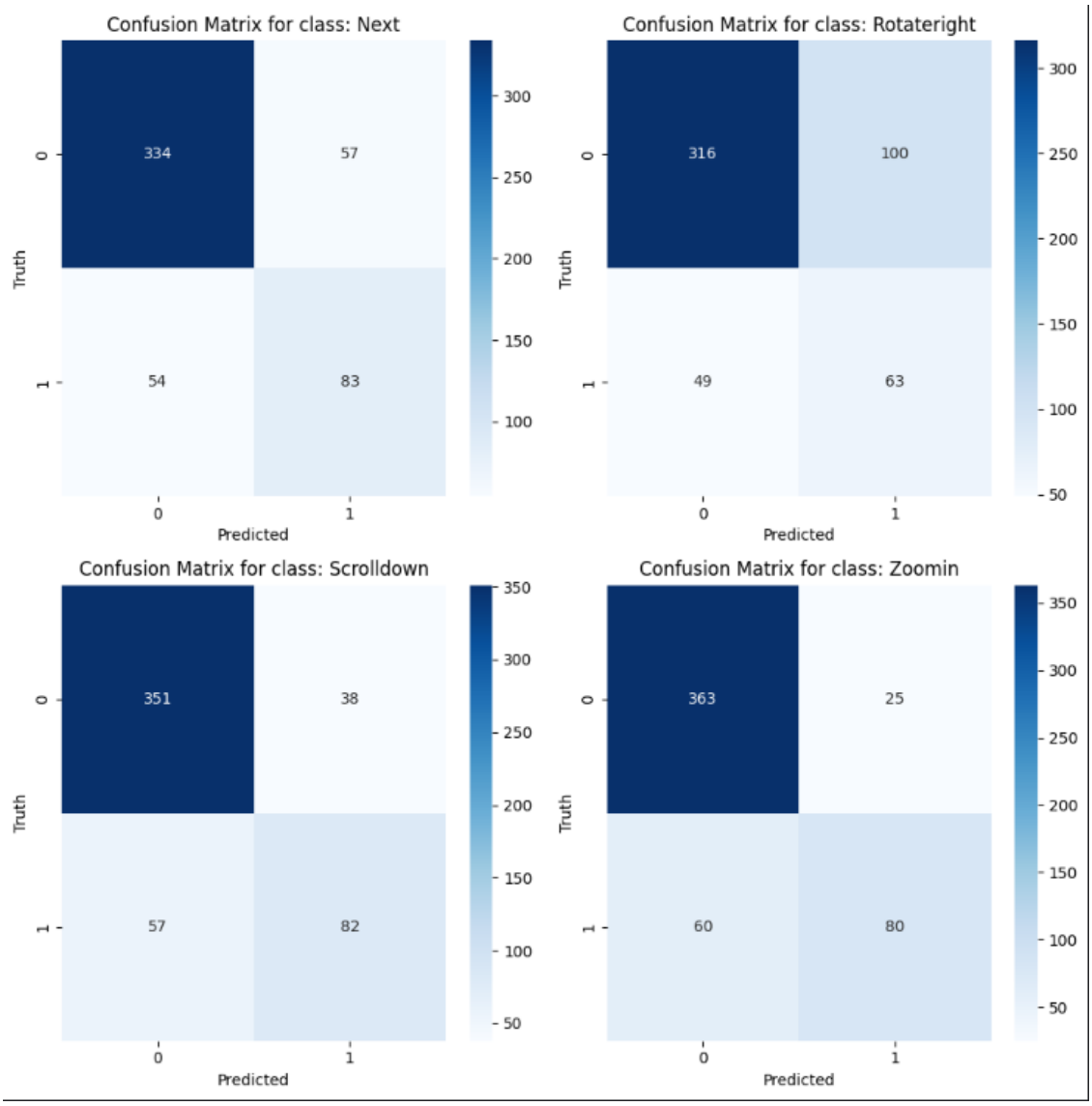
The hypothesis was that the low batch size of 5 was causing this issue. To test this theory, the batch size was increased to 30. However, due to memory limitations, the sizes of the hidden layers had to be reduced to 512 and 256. These adjustments resulted in the following model:

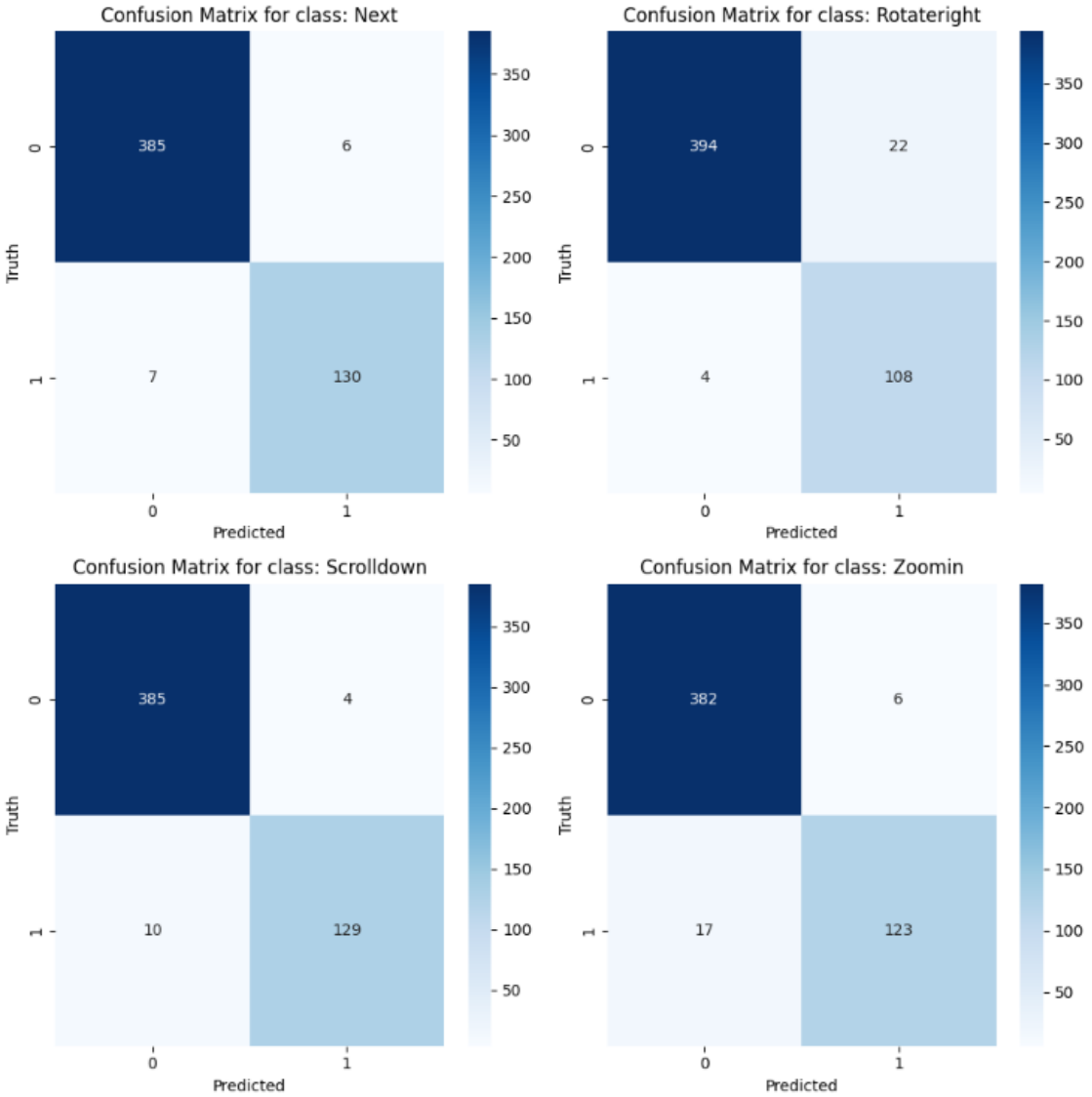
[ ]:
```
EPOCHS = 150
BATCH_SIZE = 30

# Create a sequential model
model = keras.Sequential(name="rnn_v3")

model.add(layers.LSTM(512, return_sequences=True, kernel_regularizer=l2(0.01),␣
 ↪input_shape=(X_train_reshaped.shape[1], X_train_reshaped.shape[2])))
model.add(layers.LSTM(512, return_sequences=True, dropout=0.2))
model.add(layers.LSTM(256, return_sequences=False, dropout=0.2))
model.add(layers.Dense(4, activation='softmax'))

model.build((X_train_reshaped.shape[1], X_train_reshaped.shape[2]))

# Compile the model
model.compile(
    optimizer=optimizers.SGD(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=[
        "accuracy",
        metrics.Precision(),
        metrics.CategoricalCrossentropy()
        ]
    )

model.summary()

# Train the model
model.fit(
    X_train_reshaped,
    y_train_encoded,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_reshaped,
        y_val_encoded
        ),
    callbacks=callbacks
    )

print("\nFinal evaluation")
model.evaluate(
    x=X_test_reshaped,
    y=y_test_encoded,
    return_dict=True
    )
```

```
# Kaggle only
model.save(f"/kaggle/working/trained_models/rnn_model_35_100x100_{model.name}")
show_confusion_matrix(X_test_reshaped,model)
show_confusion_matrix(X_test_reshaped,model, True)
```

```
Model: "rnn_v3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_24 (LSTM)              (None, 35, 512)           21530624

 lstm_25 (LSTM)              (None, 35, 512)           2099200

 lstm_26 (LSTM)              (None, 256)               787456

 dense_8 (Dense)             (None, 4)                 1028

=================================================================
Total params: 24418308 (93.15 MB)
Trainable params: 24418308 (93.15 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/150
53/53 [==============================] - 13s 162ms/step - loss: 35.3383 - accuracy: 0.2473 - pr
Epoch 2/150
53/53 [==============================] - 6s 121ms/step - loss: 35.2680 - accuracy: 0.2473 - pre
Epoch 3/150
53/53 [==============================] - 6s 121ms/step - loss: 35.1892 - accuracy: 0.2720 - pre
Epoch 4/150
53/53 [==============================] - 6s 119ms/step - loss: 35.1176 - accuracy: 0.2676 - pre
Epoch 5/150
53/53 [==============================] - 7s 125ms/step - loss: 35.0497 - accuracy: 0.2568 - pre
Epoch 6/150
53/53 [==============================] - 6s 123ms/step - loss: 34.9676 - accuracy: 0.3030 - pre
Epoch 7/150
53/53 [==============================] - 6s 121ms/step - loss: 34.8992 - accuracy: 0.2796 - pre
Epoch 8/150
53/53 [==============================] - 6s 121ms/step - loss: 34.8243 - accuracy: 0.3080 - pre
Epoch 9/150
53/53 [==============================] - 6s 122ms/step - loss: 34.7506 - accuracy: 0.3112 - pre
Epoch 10/150
53/53 [==============================] - 6s 122ms/step - loss: 34.6748 - accuracy: 0.3397 - pre

Epoch 20/150
53/53 [==============================] - 7s 127ms/step - loss: 33.9518 - accuracy: 0.4073 - pre

Epoch 40/150
53/53 [==============================] - 7s 125ms/step - loss: 32.5209 - accuracy: 0.6091 - pre
```

```
Epoch 60/150
53/53 [==============================] - 7s 126ms/step - loss: 31.0066 - accuracy: 0.7812 - pre

Epoch 80/150
53/53 [==============================] - 7s 127ms/step - loss: 29.3378 - accuracy: 0.9001 - pre

Epoch 100/150
53/53 [==============================] - 6s 123ms/step - loss: 27.7825 - accuracy: 0.9855 - pre

Epoch 120/150
53/53 [==============================] - 7s 125ms/step - loss: 26.4841 - accuracy: 0.9968 - pre

Epoch 130/150
53/53 [==============================] - 6s 123ms/step - loss: 25.8973 - accuracy: 0.9987 - pre
Epoch 131/150
53/53 [==============================] - 6s 123ms/step - loss: 25.8418 - accuracy: 0.9987 - pre
Epoch 132/150
53/53 [==============================] - 6s 123ms/step - loss: 25.7865 - accuracy: 0.9987 - pre
Epoch 133/150
53/53 [==============================] - 6s 124ms/step - loss: 25.7301 - accuracy: 0.9987 - pre
Epoch 134/150
53/53 [==============================] - 6s 121ms/step - loss: 25.6714 - accuracy: 1.0000 - pre


Final evaluation
17/17 [==============================] - 0s 26ms/step - loss: 25.8919 - accuracy: 0.5833 - prec
```


Confusion Matrix for class: Next
334
57
54
83
Truth
Predicted
Confusion Matrix for class: Rotateright
316
100
49
63
Truth
Predicted
Confusion Matrix for class: Scrolldown
351
38
57
82
Truth
Predicted
Confusion Matrix for class: Zoomin
363
25
60
80
Truth
Predicted



The increase in batch size, coupled with the reduction in the number of neurons per layer, did in fact lead to an enhancement in the model's performance, achieving nearly 60% accuracy on the test set. The true positive rates for each gesture were also evenly distributed around 60%, although the 'RotateRight' gesture had a significantly higher number of false positives compared to the other gestures.

Conclusion It can be stated that this model represents a step forward compared to previous iterations, even though it hasn't yet achieved the desired results. As the input data consists of an undefined amount of videos, each containing 35 frames with a height and width of 100, it is possible that these are simply too many features to train an rnn model (suffering from the already discussed size constraints) reliably. As a result, another approach to these first layers may be fitting

The latest model represents a step forward compared to previous iterations, despite not yet reaching the desired outcomes. Given that the input data comprises an indeterminate number of videos, each with 35 frames of 100x100 pixels, it's possible that the sheer volume of features presents too much of a challenge for reliably training an RNN model (with layer size constraints).

Consequently, alternative approaches to these initial layers could prove to be beneficial.

2.8.6 RNN model V4 - A new approach

In an attempt to solves the problems encountered by previous iterations, the initial layers of this model are now borrowed from the CNN model. The rationale behind this approach is that the CNN component will condense the input data, effectively filtering out non-essential features. This allows the RNN model to concentrate on the most crucial aspects of the data.

[ ]:
```
# This code block is an exact replica of the one defined in the CNN model.

input_shape = (None, VIDEO_LENGTH, HEIGHT, WIDTH, 1)
input = layers.Input(shape=(input_shape[1:]))
x = input

x = Conv2Plus1D(filters=16, kernel_size=(3, 7, 7), padding="same")(x)
x = layers.BatchNormalization()(x)
x = layers.ReLU()(x)
x = ResizeVideo(HEIGHT // 2, WIDTH // 2)(x)

x = add_residual_block(x, 16, (3, 7, 7))
x = ResizeVideo(HEIGHT // 4, WIDTH // 4)(x)

x = add_residual_block(x, 32, (3, 7, 7))
x = ResizeVideo(HEIGHT // 8, WIDTH // 8)(x)

x = add_residual_block(x, 64, (3, 3, 3))
x = ResizeVideo(HEIGHT // 16, WIDTH // 16)(x)

x = add_residual_block(x, 128, (3, 3, 3))
x = ResizeVideo(HEIGHT // 32, WIDTH // 32)(x)

x = add_residual_block(x, 256, (3, 3, 3))
x = ResizeVideo(HEIGHT // 64, WIDTH // 64)(x)

features = x.shape[2] * x.shape[3] * x.shape[4]
x = layers.Reshape((x.shape[1], features))(x)
```

Given that the input features have already undergone extensive processing, a GRU layer was selected over an LSTM layer to simplify the model and limit the risk of overfitting.

[ ]:
```
# "New" (recurrent) fraction of the model

x = layers.GRU(256, return_sequences=False)(x)
x = layers.Dense(4,activation="softmax")(x)
```

[ ]:
```
# Build and compile the model
```

```
model = keras.Model(input, x)

frames = next(iter(X_train))
model.build(frames)

model.summary()

model.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    optimizer=keras.optimizers.Adam(learning_rate=0.0001),
    metrics=["accuracy"]
)
```

[ ]:
```
y_train_encoded = to_categorical(y_train, num_classes=4)
y_val_encoded = to_categorical(y_val, num_classes=4)

print(f"Shapes:\n\t{model.output_shape=}\n\t{y_train.shape=}\n\t{y_val.
 ↪shape=}\n")

history = model.fit(
    x=X_train,
    y=y_train,
    epochs=20,
    validation_data=(X_val, y_val),
    callbacks=callbacks
    )
```

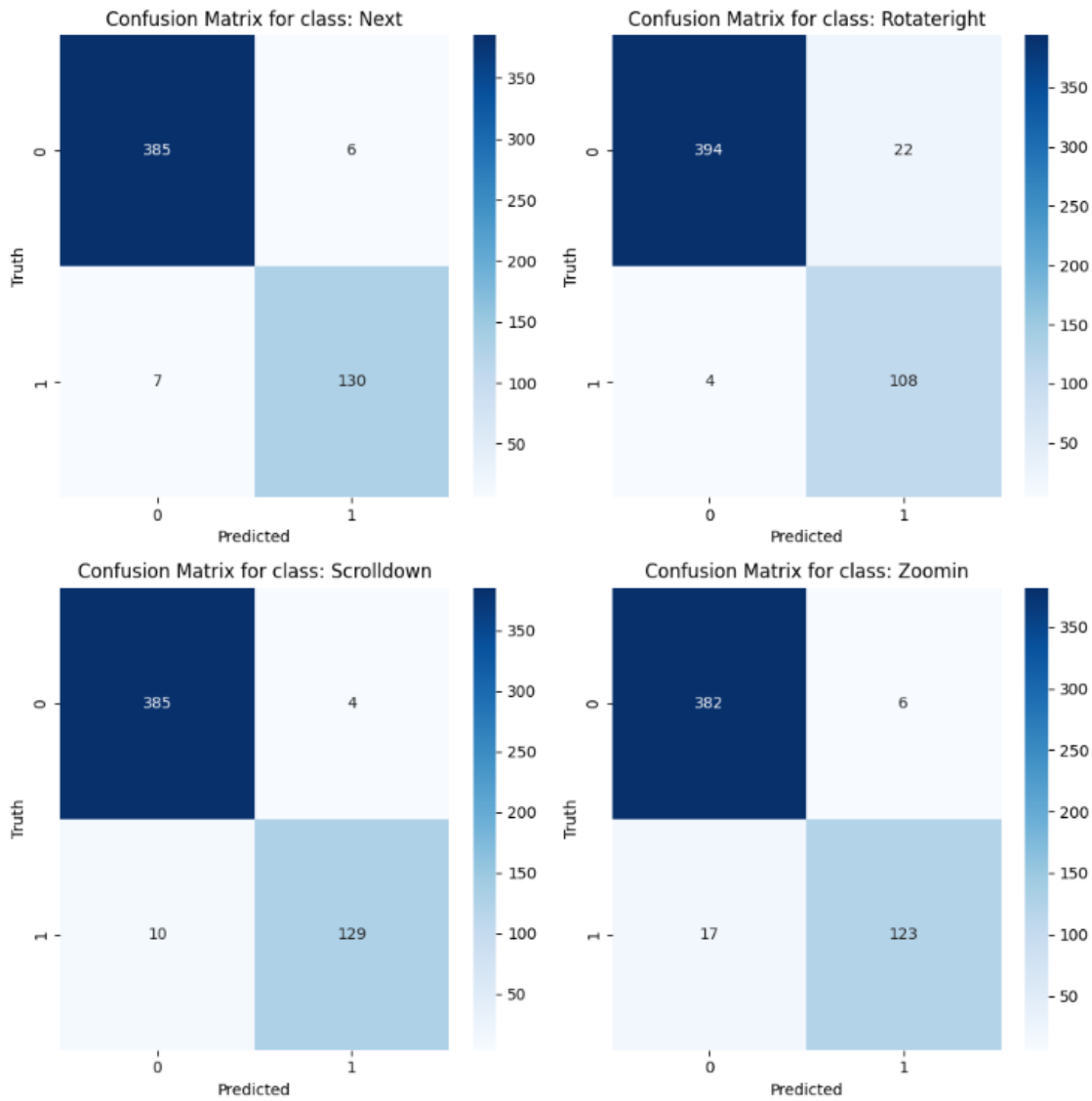
[ ]:
```
print("\nEvaluation")
model.evaluate(x=X_test, y=y_test, return_dict=True)

# kaggle only
model.save("/kaggle/working/trained_models/rnn_model_35_100x100_v4")
show_confusion_matrix(X_test,model)
show_confusion_matrix(X_test,model,True)
```

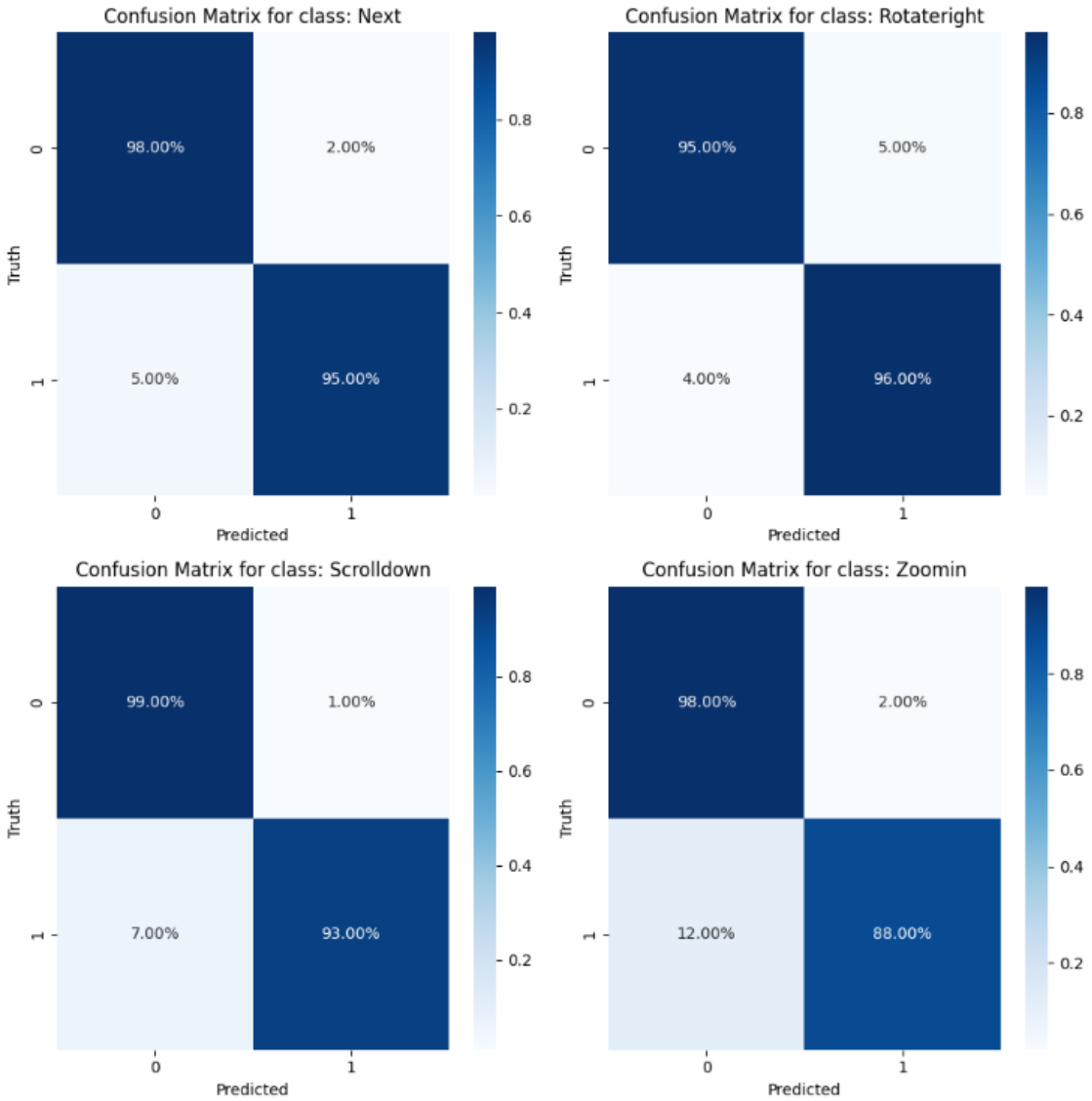
```
Model: "rnn_model_v4"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape                 Param #   Connected to
==================================================================================================
 input_2 (InputLayer)           [(None, 35, 100, 100, 1)]    0         []

 conv2_plus1d (Conv2Plus1D)     (None, 35, 100, 100, 16)     1584      ['input_2[0][0]']

 batch_normalization (Batch     (None, 35, 100, 100, 16)     64        ['conv2_plus1d[0][0]']
 Normalization)

 re_lu (ReLU)                   (None, 35, 100, 100, 16)     0         ['batch_normalization[0][0
```

```
resize_video (ResizeVideo)  (None, 35, 50, 50, 16)      0          ['re_lu[0][0]']

residual_main (ResidualMai  (None, 35, 50, 50, 16)      26752      ['resize_video[0][0]']
n)

add (Add)                   (None, 35, 50, 50, 16)      0          ['resize_video[0][0]', 'res

resize_video_1 (ResizeVide  (None, 35, 25, 25, 16)      0          ['add[0][0]']
o)

project (Project)           (None, 35, 25, 25, 32)      608        ['resize_video_1[0][0]']

residual_main_1 (ResidualM  (None, 35, 25, 25, 32)      81664      ['resize_video_1[0][0]']
ain)

add_1 (Add)                 (None, 35, 25, 25, 32)      0          ['project[0][0]', 'residual

resize_video_2 (ResizeVide  (None, 35, 12, 12, 32)      0          ['add_1[0][0]']
o)

project_1 (Project)         (None, 35, 12, 12, 64)      2240       ['resize_video_2[0][0]']

residual_main_2 (ResidualM  (None, 35, 12, 12, 64)      80384      ['resize_video_2[0][0]']
ain)

add_2 (Add)                 (None, 35, 12, 12, 64)      0          ['project_1[0][0]', 'residu

resize_video_3 (ResizeVide  (None, 35, 6, 6, 64)        0          ['add_2[0][0]']
o)

project_2 (Project)         (None, 35, 6, 6, 128)       8576       ['resize_video_3[0][0]']

residual_main_3 (ResidualM  (None, 35, 6, 6, 128)       320512     ['resize_video_3[0][0]']
ain)

add_3 (Add)                 (None, 35, 6, 6, 128)       0          ['project_2[0][0]', 'residu

resize_video_4 (ResizeVide  (None, 35, 3, 3, 128)       0          ['add_3[0][0]']
o)

project_3 (Project)         (None, 35, 3, 3, 256)       33536      ['resize_video_4[0][0]']

residual_main_4 (ResidualM  (None, 35, 3, 3, 256)       1280000    ['resize_video_4[0][0]']
ain)

add_4 (Add)                 (None, 35, 3, 3, 256)       0          ['project_3[0][0]', 'residu
```

```
 resize_video_5 (ResizeVide  (None, 35, 1, 1, 256)        0         ['add_4[0][0]']
 o)

 reshape (Reshape)           (None, 35, 256)              0         ['resize_video_5[0][0]']

 gru (GRU)                   (None, 256)                  394752    ['reshape[0][0]']

 dense_4 (Dense)             (None, 4)                    1028      ['gru[0][0]']

==================================================================================================
Total params: 2231700 (8.51 MB)
Trainable params: 2231668 (8.51 MB)
Non-trainable params: 32 (128.00 Byte)
__________________________________________________________________________________________________
Epoch 1/20
50/50 [==============================] - 100s 1s/step - loss: 1.4897 - accuracy: 0.2536 - val_l
Epoch 2/20
50/50 [==============================] - 63s 1s/step - loss: 1.3178 - accuracy: 0.3567 - val_lo
Epoch 3/20
50/50 [==============================] - 63s 1s/step - loss: 1.1415 - accuracy: 0.4725 - val_lo
Epoch 4/20
50/50 [==============================] - 63s 1s/step - loss: 0.9802 - accuracy: 0.5819 - val_lo
Epoch 5/20
50/50 [==============================] - 63s 1s/step - loss: 0.7189 - accuracy: 0.7223 - val_lo
Epoch 6/20
50/50 [==============================] - 64s 1s/step - loss: 0.5589 - accuracy: 0.8102 - val_lo
Epoch 7/20
50/50 [==============================] - 63s 1s/step - loss: 0.4704 - accuracy: 0.8412 - val_lo
Epoch 8/20
50/50 [==============================] - 63s 1s/step - loss: 0.3813 - accuracy: 0.8627 - val_lo
Epoch 9/20
50/50 [==============================] - 63s 1s/step - loss: 0.3067 - accuracy: 0.9089 - val_lo
Epoch 10/20
50/50 [==============================] - 63s 1s/step - loss: 0.2648 - accuracy: 0.9209 - val_lo
Epoch 11/20
50/50 [==============================] - 63s 1s/step - loss: 0.2054 - accuracy: 0.9431 - val_lo
Epoch 12/20
50/50 [==============================] - 63s 1s/step - loss: 0.2064 - accuracy: 0.9374 - val_lo
Epoch 13/20
50/50 [==============================] - 63s 1s/step - loss: 0.1456 - accuracy: 0.9595 - val_lo
Epoch 14/20
50/50 [==============================] - 63s 1s/step - loss: 0.1294 - accuracy: 0.9690 - val_lo
Epoch 15/20
50/50 [==============================] - 63s 1s/step - loss: 0.0952 - accuracy: 0.9760 - val_lo
Epoch 16/20
50/50 [==============================] - 63s 1s/step - loss: 0.0619 - accuracy: 0.9886 - val_lo
Epoch 17/20
50/50 [==============================] - 63s 1s/step - loss: 0.0504 - accuracy: 0.9930 - val_lo
```

```
Evaluation
17/17 [==============================] - 7s 436ms/step - loss: 0.2176 - accuracy: 0.9280
```





Kaggle When testing the model's performance with the Kaggle dataset, it achieves an accuracy of 77%. This represents a 16% decrease, suggesting that the model may be overfitting to the local data.

One potential explanation for this could be that the CNN layers, while abstracting the data, process similar videos identically. This results in the data fed to the RNN component of the model being highly similar between the training and test sets, enabling the model to match the test set with the most similar instances from the training data. Thus, when confronted with new videos, the generated abstraction may not align with any previously seen patterns, forcing the model to make educated guesses and consequently increasing the error rate.

Conclusion This hybrid model, has shown superior performance compared to previous RNN models. The model achieved an impressive accuracy, although this is largely attributable to the CNN component of the model. However, given that the RNN component has not been optimized yet, there is significant potential for further improvement. This model may even surpass the

performance of the current standalone CNN model with further tuning.

For future work, several routes could be explored to further improve this model. These include tuning the model's optimizer, such as replacing the stochastic gradient descent with a more advanced optimizer like Adam or AdamW, or adjusting the learning rate. The model's architecture could also be refined, drawing inspiration from the previous iterations. Implementing dropout and regularization techniques within or between layers could further help limit overfitting and improve the model's generalization capabilities. Additionally, if the goal were to achieve maximum accuracy, hyperparameter tuning techniques such as grid search or random search could be used for this.

2.9 Pipeline for testing kaggle datasets

2.9.1 Load kaggle dataset

[ ]:
```
def get_kaggle_id(path: str) -> list:
    ids = []
    video_dirs = sorted(os.listdir(path), key=numericalSort)
    for video_dir in video_dirs:
        ids.append(str(video_dir))
    return ids
```

[ ]:
```
shape = OutputShape(None, 35, 100, 100)
KAGGLE_DATASET_PATH = "./merged/"

kaggle_videos = load_dataset(KAGGLE_DATASET_PATH)
kaggle_ids = get_kaggle_id(KAGGLE_DATASET_PATH)
```

2.9.2 Preprocess kaggle dataset

[ ]:
```
for i, video in enumerate(kaggle_videos):
    kaggle_videos[i] = preprocess_video(video, shape)

kaggle_videos = np.array(kaggle_videos)
```

2.9.3 Load trained model

[ ]:
```
loaded_model = keras.models.load_model("./trained_models/
↪cnn_model_35_100x100_v9/")
```

2.9.4 Make predictions

[ ]:
```
def save_predictions_to_csv(ids: list[str], predictions: list[str], file_name:␣
↪str):
    # Save all ids and labels row by row in a csv file (id, label)
    print(len(ids), len(predictions))
    df = pd.DataFrame({"ID": ids, "Label": predictions})
    df.to_csv(file_name, index=False)
```

[ ]:
```
predictions = loaded_model.predict(kaggle_videos)
predicted_labels = np.argmax(predictions, axis=-1)
save_predictions_to_csv(kaggle_ids, predicted_labels, "./kaggle_predictions/
↪cnn_model_35_100x100_v8.csv")
```